Mitigating Carbon Traps in Carbon-Aware Workflow Scheduling under Real Carbon-Intensity Forecasts

Hemn Khezri [a,*], Mohammad Trik [b]

[a] *Young Researchers and Elite Club, Sardasht Branch, Islamic Azad University, Sardasht, Iran*

[b] *Department of Computer Engineering, Bou.C., Islamic Azad University, Boukan, Iran*

[*] Corresponding author. E-mail: hemnkhezri21@gmail.com

Abstract

Carbon-aware workflow schedulers defer tasks into periods of cleaner electricity, but they are usually evaluated with perfect knowledge of future carbon intensity. We replay deadline-constrained DAG workflows against the carbon-intensity forecasts that the Great Britain grid operator actually issued, in a sequence of pre-registered studies with list and local-search schedulers, whole-workflow shifting and the official code of CaWoSched and CWM. In the confirmatory study, issued forecasts kept 68% [62%, 74%] of the oracle saving; this loss was significant under the sealed analysis but not after clustering by week or correcting for multiple tests. About one plan in seven with moderate slack emitted more than a carbon-agnostic HEFT schedule; we call these outcomes carbon traps. A sealed extension to four further grids with model forecasts found a similar pooled loss (66% kept), but the share kept ranged from 19% to 85% across grids, in line with the ratio of forecast error to short-term variability (exploratory). A gate calibrated on errors of forecast differences cut traps by 8–9 percentage points while keeping 84–87% of the saving. With runtime variability, carbon-aware plans on GB missed 31–49% of deadlines, against at most 1.5% for HEFT. In a robustness study, reserving a quarter of the slack cut misses under moderate noise to under 4% while keeping 81% of the saving, no simpler gate dominated ours, and re-planning, quantile-based and scenario-based planning did not reduce traps further. Whole-workflow shifting matched task-level scheduling. Evaluations of carbon-aware schedulers should use issued forecasts and report traps and deadline misses.

Keywords: carbon-aware computing; workflow scheduling; carbon-intensity forecasts; carbon traps; deadline-constrained DAG; forecast uncertainty; pre-registration

1. Introduction

Data centres used about 1–1.5% of global electricity around 2020 [1], and their demand is now growing quickly with artificial intelligence [2]. Regulators have begun to require operators to report energy use and its sources [3]. Carbon-aware computing reduces emissions without new hardware: because the carbon intensity of grid electricity varies by hour and by place, flexible work can run when or where electricity is cleaner [4,5]. Scientific workflows are natural candidates. They are large graphs of batch tasks (DAGs), and users often accept a deadline later than the shortest possible completion time. The slack between the two can be used to move tasks into cleaner periods.

A recent line of work schedules workflows with exactly this aim. CaWoSched shifts the tasks of a fixed mapping to reduce the energy drawn above a time-varying green power budget [6]. CWM maps and schedules jointly on heterogeneous clusters and reports a median reduction of its carbon cost of 69% relative to HEFT when the deadline is twice the makespan [7]. Precedence-aware schedulers for data-processing clusters defer non-critical tasks during high-carbon periods [8], and trace-driven studies estimate savings above 80% for workflows that can wait for days [9]. These evaluations share one assumption: the scheduler knows the future carbon intensity, or a budget derived from it, exactly.

In practice a scheduler can know only a forecast. Grid operators and research groups publish forecasts from a few hours to four days ahead [10,11], and their errors grow with the horizon. Studies of forecast error in carbon-aware computing either add synthetic noise to the actual signal [4,12], or use forecasts that a model produced after the fact [13,14]. They also consider independent jobs. Workflows differ. A workflow plan contains hundreds of deferral decisions made on the same forecast, a deferred task delays its successors, and all tasks share one deadline. A forecast error can therefore turn a plan that looks greener than the carbon-agnostic one into a plan that emits more. We call such an outcome a **carbon trap**. Deferral also uses up the slack that would otherwise absorb tasks that run longer than planned.

This paper asks four questions. How much of the oracle saving of carbon-aware workflow schedulers survives the forecasts that were actually issued? How often do their plans become carbon traps, and can traps be reduced cheaply? Does re-planning on newer forecasts remove the problem? And what happens to deadlines when task runtimes vary? We answer them in a sequence of pre-registered studies. The main study replays 3,000 WfCommons workflow jobs [15] against the 48-hour forecast vintages published by the Great Britain grid operator, archived as they were issued. It compares oracle intensity, synthetic noise and issued forecasts for our list and local-search schedulers, whole-workflow shifting and the official code of CaWoSched. Three further studies extend the evaluation. The first covers four grids in the United States and Germany with model forecasts and adds the official code of CWM (4,800 jobs). The second uses a heterogeneous platform with idle power and CWM's own processor types (1,600 jobs). The third, a robustness study, tests re-planning, a work-conserving executor, slack reservation and alternative gates (1,200 jobs). Fig. 1 gives an overview of the evaluation framework and of the components we propose.

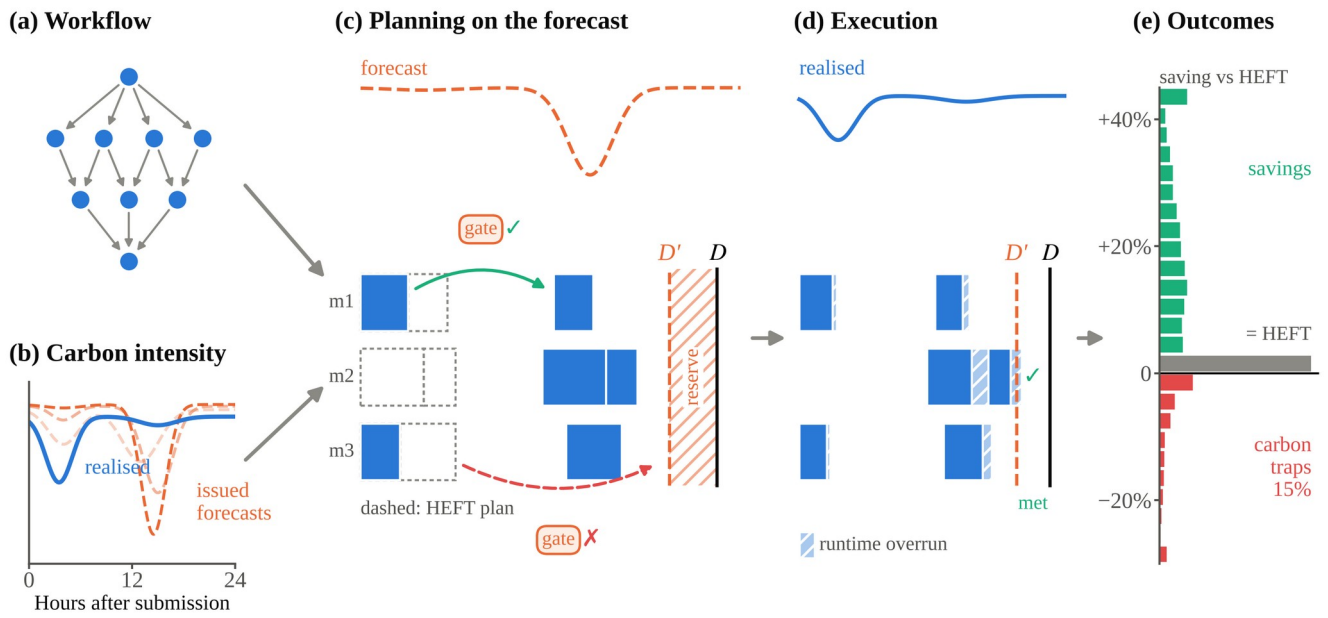


Fig. 1. Overview of the evaluation and of the proposed components. (a) Workflow (DAG) with deadline D. (b) Realised intensity and issued forecast vintages. (c) A plan built on the forecast (filled) against HEFT (dashed); the gate allows (green) or blocks (red) deferrals, and the slack reserve plans against D′ < D. (d) Execution with runtime overruns. (e) Per-run savings of CA-LS under issued GB forecasts (f ≥ 1.5); red: carbon traps. The schematic panels and the plotting code were prepared with the assistance of Claude (Anthropic, Claude Opus 5.5).

In short, the answers are as follows. Issued forecasts kept about two-thirds of the oracle saving on GB, and model forecasts kept a similar share pooled over four other grids, but from a fifth to most of it depending on the grid. About one plan in seven with moderate slack became a carbon trap. Across grids, the loss was associated with the forecast error relative to the short-term variability of intensity. Re-planning on newer forecasts improved optimisation but added little information and did not reduce traps. A gate on forecast-difference errors reduced traps on every grid and platform at a modest cost in saving. Simple whole-workflow shifting did as well as task-level scheduling under forecasts. With runtime variability, carbon-aware plans missed a third to a half of their deadlines. Reserving slack removed most of these misses; a work-conserving executor did not. On a heterogeneous platform the loss was smaller, because much of the saving came from energy-aware mapping, which needs no forecast; the part due to timing was too small there to measure how much of it forecasts lose.

The paper makes four contributions.

- To our knowledge, the first evaluation of carbon-aware DAG schedulers, including the official CaWoSched and CWM codes, against forecast vintages that were actually issued, rather than oracle or synthetic signals.
- Evidence from five grids that forecast error turns a sizeable share of workflow plans into carbon traps, that re-planning on newer forecasts does not remove them, and that across grids the loss is associated with the forecast error relative to the short-term variability of carbon intensity.
- A lightweight gate calibrated on forecast-difference errors, with a per-decision bound (Proposition 1, exact for single-window deferrals and approximate for longer tasks) and empirical coverage at or above nominal, that reduces traps on every grid; evidence that more elaborate planners (re-planning, planning on an upper quantile, scenario-based risk control) do not reduce traps further; and evidence that a simple slack reserve removes most of the deadline risk of carbon-aware plans.
- A transparent protocol with sealed analysis plans, a decision timeline, logged deviations and open code.

Section 2 reviews related work. Section 3 defines the model and outcomes, Section 4 the schedulers and the gate, and Section 5 the data and study design. Section 6 reports the results, Section 7 discusses their implications and limitations, and Section 8 concludes.

2. Background and related work

2.1 Shifting independent workloads, with and without forecast uncertainty

Early work on geographical load balancing routed requests to regions with cleaner or cheaper power [16–18], and green datacentres matched deferrable load to on-site renewable supply [19]. Wiesner et al. [4] made temporal shifting concrete for the cloud, with trace-driven savings of up to about a third for flexible nightly jobs; Gaussian forecast noise of 5–10% cost them one to two percentage points. Sukprasert et al. [12] showed with 123 regions that realistic slack yields much smaller savings than idealised settings, and that a forecast error of about 14% MAPE raises emissions by around 3%. A companion study found that optimising for average or for marginal intensity can lead to opposite decisions [20]. Google's production system sets hourly capacity limits for flexible batch work from day-ahead forecasts, inflating demand by a high quantile of recent forecast errors [5]. Related systems scale resources with carbon intensity [21], manage real-time workloads on renewable-driven cores [22], migrate tasks across datacentres in space and time [23], trade carbon against cost [24] or exploit spatial variation at the edge [25–27]; surveys organise the field [28,29], and system-level studies caution that flexible datacentres do not always lower emissions [30].

A second line of work designs algorithms that tolerate uncertain signals, again for a single deferrable job. Online pause-and-resume algorithms decide from the current signal and its known bounds [31]. Learning-augmented algorithms add predictions while keeping worst-case guarantees [32–34], and algorithm selection picks the best heuristic per region [35]. Closest to our work, Li et al. [13] wrap CarbonCast forecasts in sequential conformal intervals. They report that point forecasts pick the wrong hours on 10.6–16.8% of days, raising emissions by 2.7–7.3%, and that acting on the intervals avoids most of this increase. Learned schedulers that integrate carbon forecasts into workload scheduling have also been proposed [36]. These studies establish that forecast error matters, but for independent jobs, and with forecasts produced by a model after the fact.

2.2 Carbon-aware scheduling of DAGs and workflows

Workflow scheduling has a long history in deadline- and cost-constrained cloud settings [37–39], with HEFT as the standard list-

scheduling reference [40]. Environment-conscious scheduling of HPC applications [41] and renewable-aware multi-cloud placement [42] introduced carbon as an objective. CaWoSched [6] fixes the mapping and processor order and shifts start times to minimise the energy drawn above a green power budget under a hard deadline. CWM [7] maps and schedules jointly on heterogeneous clusters with idle power. PCAPS [8] defers non-critical Spark tasks during high-carbon periods, and Bostandoost et al. [43] compute exact bounds for small DAG instances as a job-shop problem. West et al. [9] quantify shifting, interruption and scaling for seven Nextflow workflows and find large savings when workflows can wait for days; their sensitivity analysis with 5–15% synthetic forecast error shows small shortfalls. With that exception, these studies assume the future signal is known, and only West et al. vary task runtimes, in a post hoc check.

2.3 Carbon-intensity forecasts

The GB operator issues a 48-h national forecast every 30 minutes [10]. Research forecasters extend the horizon: DACF predicts one day ahead [44], CarbonCast up to 96 h with regional MAPEs of a few percent to about 20% [11], EnsembleCI improves on CarbonCast by about a fifth [45], and LiteCast trains on one week of history [14]. LiteCast's authors show that a forecast's ranking of hours, rather than its point accuracy, determines the saving of a single job. Forecasters are usually judged by accuracy metrics, not by the decisions they induce, and scheduling studies usually regenerate forecasts with a model rather than replaying what an operator published.

2.4 Robust DAG scheduling and conformal calibration

Robust DAG scheduling studies plans under uncertain durations; slack absorbs overruns, and makespan variability trades off against it [46–49]. Carbon-aware schedulers spend exactly this slack, but they have not been evaluated under runtime variation. Conformal prediction gives distribution-free error bounds from a calibration set [50], extended to time series [51] and to distribution shift [52]. Our gate uses an empirical quantile of recent errors in this spirit, but of forecast differences rather than levels, because a deferral compares two windows.

2.5 Positioning

Table 1 compares the closest studies with ours. No prior study combines deadline-constrained DAG scheduling, forecasts that were actually issued, runtime variability, the official code of the closest published schedulers and a pre-registered evaluation.

Table 1. Closest related work and this study. "Synthetic" denotes noise added to actual intensity; "model forecasts" are produced by a forecasting model after the fact, as opposed to vintages issued at the time.

Work	Workload	Deadline	Carbon information at decision time	Effect of forecast error measured	Runtime noise	Other published schedulers run from their official code
Wiesner et al. [4]	Batch and ML jobs	Window	Oracle; synthetic Gaussian (5–10%)	Yes (synthetic)	No	No
Sukprasert et al. [12]	Batch and interactive jobs	Slack	Oracle; synthetic uniform (0–100%)	Yes (synthetic)	No	No
Radovanović et al. [5]	Batch jobs (production)	24 h	Operational day-ahead forecasts	No	No	n/a
Li et al. [13]	Power trace	No	Model forecasts (CarbonCast) with conformal intervals	Yes	No	No
Joseph et al. [14]	Batch jobs	Slack	Model forecasts (LiteCast, CarbonCast)	Yes	No	No
Learning-augmented single-job algorithms [32–34]	Single job	Yes	Predictions with uncertainty; synthetic advice or error	Yes	Job length (LACS)	No
PCAPS [8]	DAG (Spark)	No	Online, current CI with known bounds	No	No	No
Bostandoost et al. [43]	DAG (batch)	Makespan stretch	Oracle	No	No	No
West et al. [9]	Scientific workflows	No	Oracle; synthetic error (5–15%)	Yes (synthetic)	Post hoc	No
CaWoSched [6]	Workflow, fixed mapping	Yes	Known green budget	No	No	No
CWM [7]	Workflow	Yes	Known green budget	No	No	Yes (CaWoSched)
This study	**Workflows**	**Yes**	**Oracle; synthetic; issued vintages (GB); model forecasts (4 grids)**	**Yes, pre-registered**	**Yes**	**Yes (CaWoSched, CWM)**

3. System model and problem

Table 2 lists the main notation.

Table 2. Main notation.

Symbol	Meaning
$G = (V, E)$; d_i	Workflow DAG; nominal duration of task i (min)
P; p	Number of machines; busy power per machine (kW)
M; D; f	HEFT makespan; deadline $D = \text{round}(f \cdot M)$; deadline factor
t_0; s; s^H	Submission time; a plan (start times); the HEFT plan
I(t); $\hat{I}(t)$	Realised carbon intensity; intensity used for planning
F; A	Forecast and realised intensity of a window; $\bar{F}$, $\bar{A}$: means over a task's duration
C(s); S(s)	Realised emissions of plan s; saving relative to HEFT
R	Share of the oracle saving kept under forecasts (Eq. 3)
e(a, b)	Difference error of a deferral from start a to start b (Eq. 5)
ℓ; h(a, b)	Deferral lag $\lvert b - a \rvert$; forecast horizon of a deferral, $\tau_c + \max(a, b) + d/2$ (Section 4.3)
τ_c	Age of the vintage at submission: t_0 minus its capture time
q[ℓ, h]; α	Gate threshold for lag bin ℓ and horizon bin h; its quantile level
κ	Forecast error relative to short-term variability (Eq. 4)
D′; ρ	Planning deadline with slack reserve, $D' = M + \rho(D - M)$; reserve parameter

3.1 Workflows, platform and deadlines

A workflow is a DAG $G = (V, E)$ of tasks with precedence constraints; task i has a duration d_i in minutes. In the main study the platform is a dedicated cluster of $P = 8$ identical machines. Each runs one task at a time and draws a constant busy power $p = 0.2$ kW; data-transfer time is ignored. On this platform HEFT reduces to list scheduling by bottom level, which we still call HEFT for consistency with the literature. Section 5.5 replaces the platform with a heterogeneous one with idle power. A workflow is submitted at time t_0 and must finish within a deadline $D = \text{round}(f \cdot M)$ measured from t_0, where M is the HEFT makespan and $f \in \{1.2, 1.5, 2.0, 3.0\}$ is the deadline factor [37,38]. A plan assigns a start time s_i to every task, at one-minute resolution. It is feasible if precedence holds, at most P tasks run at once and every task finishes by D.

3.2 Carbon cost, saving and traps

Let I(t) be the realised average carbon intensity (gCO_2/kWh), published in 30-minute windows. The operational emissions of a plan s, with p in kW, I in gCO_2/kWh and t in minutes (hence the factor 1/60), are

(1)

On the homogeneous platform idle power is not attributed; Section 7.4 discusses this assumption. The saving of a plan is measured against the HEFT plan s^H on the same instance and submission, always with the realised intensity:

(2)

A run is a **carbon trap** if $C(s) > 1.01 \cdot C(s^H)$. The 1% margin excludes ties and rounding; we also report thresholds of 0%, 2% and 5% and the expected excess emission. Traps matter even when the mean saving is positive: a carbon-aware service that makes some jobs dirtier than the carbon-agnostic default breaks its promise to the users of those jobs. We therefore treat the trap rate as a measure of tail risk alongside the mean saving. Fig. 2 shows a trap from the confirmatory data. The issued vintage put the cleanest hours 8–11 h after submission, but the realised intensity was lowest 3–7 h after it and rose afterwards; ungated CA-LS deferred the work, expected a 43% saving and emitted 23% more than HEFT, while the gate blocked the long deferral and realised an 18% saving, close to the 17% of CA-LS planned on the realised intensity. The **relative loss** of a scheduler is

(3)

the share of the oracle saving that survives forecasts, computed as a ratio of means over runs with $f \geq 1.5$ (at $f = 1.2$ the savings are small and the ratio unstable). We also report R per f, the absolute loss in percentage points and an emission-weighted ratio.

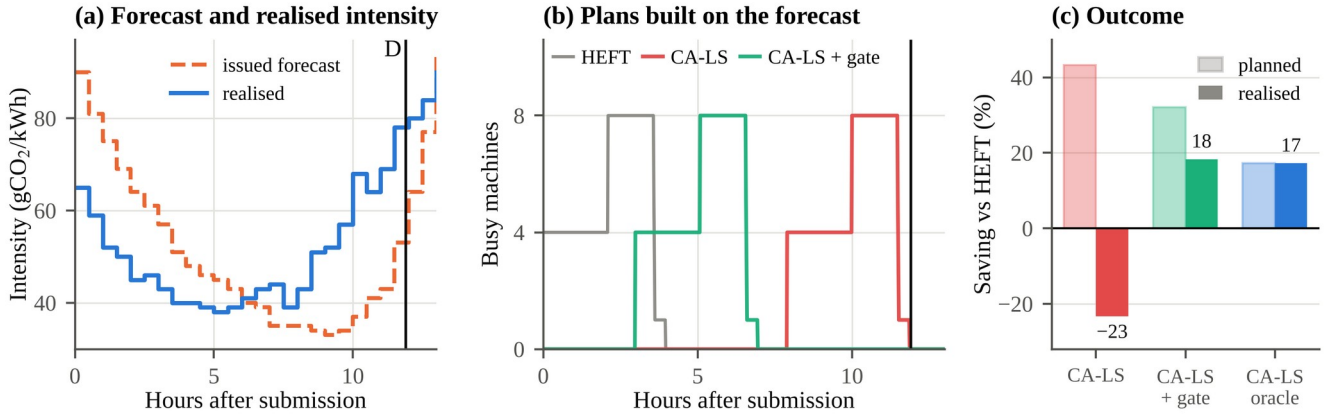


Fig. 2. A carbon trap from the confirmatory data (Epigenomics, T = 4 h, f = 3.0, submitted on 25 July 2026 at 08:00 BST, seed 11). (a) Issued forecast and realised intensity. (b) Busy machines in the HEFT plan and in the CA-LS plans without and with the gate; D: deadline. (c) Saving relative to HEFT, as planned and as realised.

3.3 Information regimes

A scheduler plans on a signal $\hat{I}$ and is evaluated on I. **Oracle:** $\hat{I} = I$. **Synthetic noise:** $\hat{I} = I \cdot (1 + \varepsilon)$, $\varepsilon \sim N(0, 0.20^2)$ independently per window; $\sigma = 0.20$ gives a median absolute relative error of $0.674\sigma \approx 13.5\%$, matching the 13.7% of 0–36-h GB forecasts in the development period. **Issued forecasts (GB):** the latest NESO vintage captured at or before $t_0 - 30$ min, i.e. a forecast an operator could have used. **Model forecasts (other grids):** out-of-sample CarbonCast forecasts, issued daily at 00:00 UTC (Section 5.4).

3.4 Runtime variability and execution

To test how plans that use the slack behave when tasks overrun, every plan is also executed with realised durations $r_i = d_i \cdot \exp(\eta_i)$, $\eta_i \sim N(0, 0.2^2)$ i.i.d. We use two execution rules, the same for every scheduler including HEFT. The **static executor** dispatches tasks in order of planned start; each starts at the latest of its planned start, its predecessors' completion and the first time a machine is free. The **reactive executor** (used in the robustness study) is work-conserving after a delay: a task waits for its planned start only while all its ancestors are on time; once an ancestor finished late, the task starts as soon as it is ready. A run misses its deadline if its realised makespan exceeds D.

3.5 Forecast error relative to variability

For the exploratory analysis of Section 6.6 we summarise a grid and period by the ratio of the forecast's mean absolute error over horizons of 0–47 h to the mean standard deviation of the realised intensity within the same 47-h windows, which we call short-term variability:

(4)

Here n runs over the N forecast values with lead times of 0–47 h issued for the grid and period, F_n and A_n are the forecast and realised intensity of value n, k runs over the K vintages, and $SD_k(A)$ is the standard deviation of the realised intensity within the 47-h window of vintage k. κ compares the error of the forecast with the variation that scheduling exploits; $\kappa \geq 1$ means the forecast error is as large as the signal.

4. Schedulers and the difference-error gate

All schedulers share one simulator, so differences between them come from their decisions.

4.1 Reference and carbon-aware schedulers

HEFT orders tasks by decreasing bottom level and starts each as early as precedence and capacity allow [40]. **Shift** keeps the HEFT plan and moves it as a whole by the offset $\delta \in [0, D - M]$ (one-minute steps) with the lowest planned cost; $\delta = 0$ wins ties. **CA-List** visits tasks in HEFT order; for each it finds the earliest feasible start a and the latest start $LST_i = D - bl_i$, where bl_i is the bottom level including d_i. Among the starts in $[a, LST_i]$ at which a machine is free, it takes the one with the lowest planned cost, and defers only if that start is strictly cheaper than a; if $a > LST_i$ the interval is empty and the task starts at a. Because LST ignores capacity, a plan can occasionally overrun D. CA-List is then re-run with a tighter internal deadline found by bisection over [M, D] (six steps); the bisection keeps the largest internal deadline whose plan meets D, not the cheapest plan, since list scheduling need not be monotone in the internal deadline. HEFT is used if no step yields a feasible plan. **CA-LS** adds $2|V|$ local-search moves to whatever plan CA-List returns, including the HEFT fallback; each moves a random task to the cheapest start allowed by its predecessors, successors, capacity and D, earlier or later than its current start. CA-LS is the primary scheduler.

We also run the official code of the two closest published schedulers without algorithmic changes: **CaWoSched** [6,53], with the HEFT mapping and the better of its slackR-LS and pressR-LS variants, and **CWM** [7,54], local-search variant. Both minimise brown energy above a green power budget. We map intensity to a budget by a linear min–max inversion over the windows up to D, from the total idle power in the dirtiest window to the total idle power plus a fraction frac of the total dynamic power in the cleanest (Supplementary Section S1 gives the formula). We used frac = 1.0, which gave both tools their best oracle saving on development data; the original authors use 0.2–0.4, which we also test (Section 6.7). We report these tools on emissions and on their native objective, the brown energy $B(s) = \Sigma_t \max(0, \text{power}(s, t) - \text{budget}(t))\, \Delta t$ over one-minute steps t ($\Delta t = 1$ min, so B is in power-minutes), computed with the budget derived from the realised intensity and evaluated on the final, safeguarded plan. Adapter details are in Supplementary Section S1.

4.2 HEFT safeguard

Every carbon-aware scheduler is wrapped in the same safeguard: its plan is kept only if it is cheaper than the HEFT plan on the planning signal. Under the oracle this makes traps impossible, which we check. Under forecasts it cannot prevent traps, because it trusts the signal that misleads the scheduler. Traps under forecasts are therefore attributable to forecast error alone. Under the oracle the safeguard kept the HEFT plan in 24% of CA-LS runs (30% at f = 1.2, 14% at f = 3.0), where local search found no cheaper plan; these runs count as zero saving.

4.3 The difference-error gate

A deferral from start a to start b pays off if the realised intensity over [b, b + d] is lower than over [a, a + d]. The scheduler sees the forecast gain; what matters is the realised gain. The error of the decision is the error of a difference,

(5)

where $\bar{F}$ and $\bar{A}$ are mean forecast and realised intensities over the task's duration. Because forecast errors in nearby windows are correlated, much of the level error cancels in short deferrals. Fig. 3 illustrates the error of a deferral decision, the calibrated thresholds and the resulting decision rule on development data. For a 30-minute deferral the 0.9-quantile of e is 7 gCO_2/kWh; it rises to 36 at 6 h and stays at 40–50 for longer lags. A rule that compares interval bounds on levels, in the style of Li et al. [13], requires a gain of 77 at every lag. It therefore blocks many safe short deferrals.

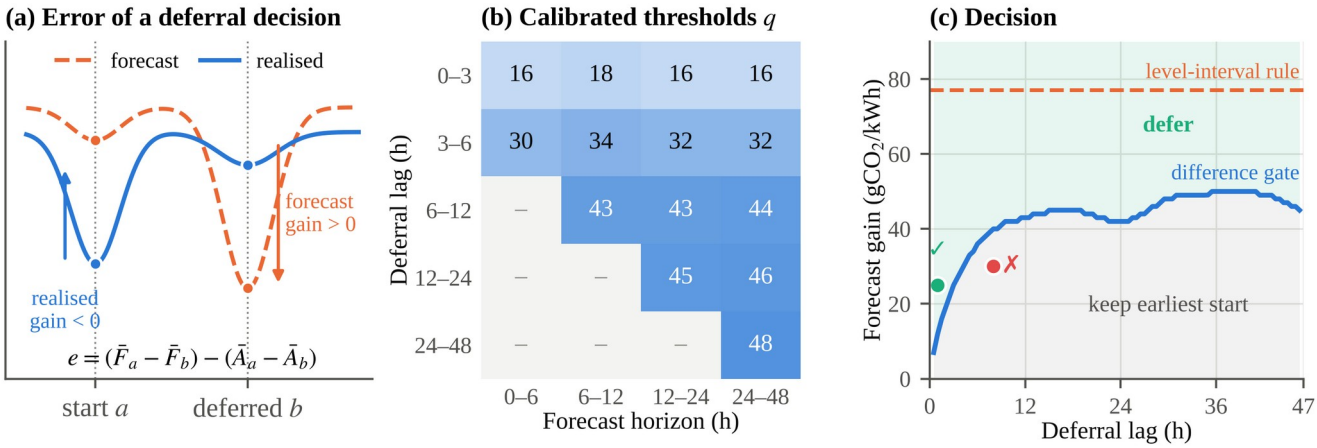


Fig. 3. The difference-error gate. (a) The error e of a deferral from a to b: the forecast gain minus the realised gain. (b) Calibrated 0.9-quantiles of e per lag × horizon bin (GB development data, Aug – Oct 2023); empty cells are impossible. (c) A deferral is allowed only above the gate's threshold (blue, pooled over horizons, Mar 2023 – Feb 2024); the level-interval rule (orange) needs 77 gCO_2/kWh at every lag. Panel (a) and the plotting code were prepared with the assistance of Claude (Anthropic, Claude Opus 5.5).

For each submission the gate collects the difference errors of all vintages captured between 7 days and 49 hours before t_0, for all pairs of windows already realised before t_0, in both directions. Calibration works on single 30-minute windows. For windows $u < v$ of a vintage captured at time c, the lag is $v - u$ and the horizon is $v - c$, the lead time of the later window. The errors are binned by lag (0–3, 3–6, 6–12, 12–24, 24–48 h) and horizon (0–6, 6–12, 12–24, 24–48 h), and the gate stores the α-quantile of each bin; a bin with fewer than 50 errors takes the largest quantile of all bins. On the confirmatory data every populated bin held at least 1,008 errors (median 17,236). At decision time a deferral from start a to start b of a task of duration d has lag $\ell = |b - a|$ and horizon $h(a, b) = \tau_c + \max(a, b) + d/2$, where τ_c is the age of the vintage at submission and a, b are measured from t_0; h is the lead time of the midpoint of the later placement. The last horizon bin is open-ended: bin(·) maps every value above 48 h to it, in calibration and at decision time alike. This matters only for the 96-h CarbonCast vintages (Section 5.4); lags never exceed the largest deadline, 36.7 h. A deferral is allowed only if the forecast mean gain exceeds $q[\ell, h]$; among allowed candidates the one with the largest margin is taken (Algorithm 1). The reference start a differs by scheduler. CA-List compares every candidate with the task's earliest feasible start. A CA-LS move compares it with the task's incumbent start s_i, which may already be deferred, and in either direction; if no candidate passes, the task keeps s_i. The only parameter, $\alpha = 0.9$, was selected on development data from {0.5, …, 0.9} by a rule fixed before tuning: the highest saving among values whose pooled trap rate is at most 5%. The gate is an empirical quantile threshold in the spirit of split-conformal calibration, and Proposition 1 states what it guarantees.

Proposition 1. Consider a deferral from a to b with forecast gain $g = \bar{F}_a - \bar{F}_b$, whose difference error e falls in bin (ℓ, h). Let q be the $\lceil (n + 1)\alpha \rceil$-th smallest of the n calibration errors of that bin. If e and the calibration errors are exchangeable (which, since the calibration errors are single-window errors, presumes a task of one window), then P(the deferral is allowed and increases the realised emissions) $\le 1 - \alpha$. *Proof.* The deferral increases emissions if and only if the realised gain $g - e$ is negative, that is, if $e > g$. It is allowed only if $g > q$. Both events together imply $e > q$, and for exchangeable errors $P(e > q) \le 1 - \alpha$ by the standard split-conformal argument [50]. □

Three gaps separate the proposition from the implementation. The code uses the empirical α-quantile rather than the order statistic, which with at least 1,008 errors per bin changes coverage by less than 0.001. The thresholds are calibrated on single windows, while most tasks span several; for them the proposition is an approximation, but averaging over a duration shrinks the error, so the thresholds are conservative (coverage 89.8% for 30-min and 95.3% for 8-h tasks; Section 6.4). And in CA-LS the bound applies to each move relative to the incumbent start, not to the total displacement of a task. The bound also holds per deferral, not per plan. A plan makes many deferrals, so a trap remains possible when several allowed deferrals err in the same direction, and serial dependence weakens exchangeability; this is why the gate reduces traps rather than removing them.

Algorithm 1. Gated deferral decision for one task, and calibration of the gate

Input: task i of duration d; reference start a (CA-List: earliest feasible start; CA-LS: incumbent start s_i); candidate starts Ω, each with a free machine and respecting precedence and D; forecast F from a vintage of age τ_c ($\bar{F}_a$: mean of F over [a, a + d)); thresholds q ← Calibrate(t_0)
Output: start time b* of task i

b* ← a; g* ← 0
for each b ∈ Ω **do**
    ℓ ← |b − a|; h ← τ_c + max(a, b) + d/2 ▷ *lag and forecast horizon*
    g ← $\bar{F}_a$ − $\bar{F}_b$ − q[bin(ℓ), bin(h)] ▷ *forecast gain minus threshold*
    if g > g* **then** b* ← b; g* ← g
end for
return b* ▷ *b* = a when no candidate passes the gate*

Procedure Calibrate(t_0)
errs[ℓ, h] ← ∅ for every lag bin ℓ and horizon bin h
for each vintage F captured at c ∈ [t_0 − 7 d, t_0 − 49 h] **do**
    for each pair of 30-min windows u < v with v − u < 48 h and v realised before t_0 **do**
        e ← (F(u) − F(v)) − (A(u) − A(v)) ▷ *realised intensity A*
        add e and −e to errs[bin(v − u), bin(v − c)]
for each bin (ℓ, h) **do** q[ℓ, h] ← α-quantile of errs[ℓ, h] **if** |errs[ℓ, h]| ≥ 50
set every remaining bin to the largest q[ℓ, h]; **return** q ▷ *bin(·) maps values above 48 h to the last bin*

5. Data and study design

5.1 Forecasts for Great Britain

NESO publishes a national carbon-intensity forecast every 30 minutes, covering the next 48 h in half-hourly windows, together with an estimate of the realised intensity [10]. The realised values are themselves estimates, since embedded wind and solar are modelled rather than metered; we take the last published value for each window. The public API keeps only the latest values, so we use an open archive of API snapshots (nmpowell/carbon-intensity-forecast-tracking, commit 578de5b [55]), which stores every forecast issued for each window. The archive has a gap between February 2024 and July 2026 caused by an outage of its scraper. We use March 2023 – February 2024 for development and 16 July – 28 October 2026, not opened before the confirmatory plan was sealed, for the confirmatory study. A vintage captured 30 minutes before submission covers 47 h after it, and every plan fits inside that span: the largest deadline is D = round(3.0 · M) ≤ 36.7 h, since M lies within 2% of T ≤ 12 h. The padding of the scheduler horizon logged as deviation 1 in Appendix A therefore never reaches a valid plan. Supplementary Fig. S1 shows the error by horizon: the mean absolute error rises from 12–13 gCO_2/kWh one hour ahead to about 26–27 gCO_2/kWh at 24–48 h. Mean intensity fell from 151 to 114 gCO_2/kWh between the periods, so the relative error is larger in 2026.

5.2 Workloads

Instances are generated with the WfCommons 1.6 recipes [15,56,57]: Montage (two sizes), Epigenomics, Blast, BWA, Cycles, 1000Genome, Seismology, SoyKB and SRASearch, with 195–391 tasks over all seeds. Each DAG is used twice, with runtimes scaled so that the HEFT makespan is within 2% of T = 4 h and of T = 12 h, so a seed contributes 20 workflow instances to every submission time; a job is one instance at one submission time. The instances of seed 10 are wide and shallow (depth 2–11, maximum width 71–297 tasks), and most are limited by machine capacity rather than by their critical path: the critical path is 5–74% of the makespan (Supplementary Table S1). A seed fixes the workflow instances, the submission times and the random choices of local search and runtime noise.

5.3 Staged, pre-registered design

Scheduler evaluations are vulnerable to choices made after seeing results [58], so every study was planned before its data were used [59]. A pilot on development data fixed the deadline repair and the gate. The confirmatory plan then fixed the data, seeds, schedulers, outcomes, hypotheses, decision rules and analysis script, and was sealed by a git commit before any 2026 result was computed. Two later studies (Sections 5.4–5.5) and a robustness study (Section 5.6) were planned and sealed the same way. All plans, scripts, deviation logs and raw results are in the repository. The simulator, schedulers, adapters and analysis scripts were written in Python with the assistance of Claude (Anthropic; Claude Opus 5.5 and Claude Sonnet 5); the authors reviewed the code, the confirmatory analyses were fixed in the sealed plans before the data were collected, and all code is released with the data (Data availability). Git timestamps are set by the authors and cannot by themselves prove the order of events. The repository also contains the sealed plan of a winter replication on GB (November 2026 – February 2027), whose results will be reported separately.

The confirmatory study uses seeds 10–14 with 30 submission times per seed, drawn uniformly from the 30-minute slots that have a complete vintage, actual intensity for the whole execution horizon and seven days of history for the gate (150 draws, 147 distinct times; runs that share a time share a bootstrap cluster). Each of the 3,000 jobs (5 seeds × 30 submissions × 20 instances) is run at four deadline factors with 13 scheduler–regime combinations (listed in the note to Table 3), giving 156,000 runs. The decision thresholds were set before the data on practical grounds: a loss of a quarter of the oracle saving (R < 0.75), a trap rate of at least one run in fifty (2%), a gate that keeps at least half of the saving, and a miss-rate gap of 10 percentage points against HEFT's near-zero rate. The threshold of 0.75 was fixed after the pilot, in which the loss had been larger (R = 0.50–0.69); the confirmatory test therefore asks whether a practically important loss recurs on new data. We report effect sizes with intervals so that readers can apply other thresholds. Table 3 summarises the design of all studies and Table 4 the timeline of decisions.

Table 3. Design factors of the four studies. Jobs = seeds × submissions per seed (× grids) × 20 instances; scheduler runs = jobs × 4 deadline factors × the scheduler–regime combinations listed in the note.

Factor	Main study (GB)	Generality extension	Platform sensitivity	Robustness study
Forecasts	Issued NESO vintages, 48 h, every 30 min	CarbonCast model forecasts, 96 h, daily	As extension	As main study
Grids and period	GB; dev. Mar 2023 – Feb 2024; confirmatory 16 Jul – 28 Oct 2026	CISO, DE, ERCO, PJM; Jul – Dec 2021	As extension	GB confirmatory period
Platform	8 identical machines, 0.2 kW busy, no idle power	As main study	4 slow + 4 fast machines, idle power attributed	As main study
Workflows	10 WfCommons DAGs per seed (195–391 tasks), each scaled to T = 4 h and T = 12 h: 20 jobs per submission	As main study	As main study	As main study
Deadline factors	$f \in \{1.2, 1.5, 2.0, 3.0\}$	Same	Same	Same
Regimes	Oracle, synthetic, issued	Oracle, synthetic, model forecast	Oracle, model forecast	Oracle, issued
Seeds × submissions	5 × 30	3 × 20 per grid	1 × 20 per grid	2 × 30 (seeds 10–11 of the main study)
Jobs / scheduler runs	3,000 / 156,000	4,800 / 288,000	1,600 / 76,800 (+12,800)	1,200 / 172,800 (+24,000)
Hypotheses	H1–H5 (sealed)	E1–E4 (sealed)	G1–G5 (sealed)	Q1–Q6 (plan sealed; robustness)

Note. HEFT is the reference of every run and is not counted; static and reactive executors and the runtime-noise draws are evaluated on the same plan. Supplementary Section S3 lists the scheduler–regime combinations of each study.

Table 4. Decision timeline. Every plan was committed before the corresponding results were computed; the commits are in the public repository.

Date (2026)	Step	Data used	Commit
29 Oct	Pilot plan sealed; pilot run; deadline repair fixed	Development (GB 2023–24)	caf34db, 3b22ef5
29 Oct	Gate designed; α selected by a pre-fixed rule	Development	cdbf532
29 Oct	Confirmatory plan sealed (H1–H5, analysis script)	None of 2026	55fff17
29–30 Oct	Confirmatory run (two engineering deviations logged before results were read); results	GB Jul–Oct 2026	cb5188e, 7136185
2 Nov	Adapter checks for CaWoSched and CWM; extension plan sealed (E1–E4)	Development	7f132e7, 1eb3069
2 Nov	Extension results; platform-sensitivity plan sealed (G1–G5); results	CarbonCast 2021	5846b88, 071e77f, c5f2234
4 Nov	Robustness plan sealed (Q1–Q6); robustness runs	GB confirmatory	107ae9d, ff0677e, 6486ffd
4 Nov	Units error in the re-planning loop found and logged; re-planning variants re-run before their results were reported	GB confirmatory	c91f9fb

5.4 Generality extension: four grids with model forecasts

To test generality across grids, forecasters and seasons, a second sealed study used out-of-sample day-ahead forecasts from CarbonCast [11,60] for four grids chosen before any result: CISO (California, solar), DE (Germany, wind), ERCO (Texas, high forecast error) and PJM (US mid-Atlantic, low forecast error), July – December 2021. These forecasts were produced by a research model and published with its repository; they are not operator vintages, and one vintage per day is issued at 00:00 UTC, so the forecast available at submission, the latest vintage captured at least 30 minutes before t_0, is 0.5–24 h old. Hourly values are split into two 30-minute windows, and plans use the first 47 h after t_0, as for GB. The gate and its calibration rule were kept unchanged, so only windows realised before t_0 enter calibration. Although the calibration window holds only four or five daily vintages, every bin that can hold errors had at least 168 of them, so the fallback for sparse bins was never triggered; placements more than 48 h after the capture of the vintage, possible in 66% of submissions, fall into the open-ended last bin. Because every grid receives its vintage at 00:00 UTC, the age of the forecast at a given local hour differs between grids (Supplementary Section S5). The extension uses new seeds 30–32 with 20 submissions per seed and grid (4,800 jobs), all schedulers of the main study and CWM. Its hypotheses mirror the main ones (E1–E3) and add a test of the runtime-variability finding (E4). They are evaluated pooled over grids; per-grid results are secondary.

5.5 Platform sensitivity: heterogeneous machines with idle power

A third sealed study repeats seed 30 of the extension (1,600 jobs) on a platform with CWM's own example processor types: four slow machines (speed 1, idle 1, busy 10 power units) and four fast ones (speed 2, idle 2, busy 15), with 1 unit = 0.02 kW. A task of work w runs $\lceil w / \text{speed} \rceil$ minutes. All machines stay powered until the later of D and the makespan, so idle power is attributed. HEFT becomes the insertion-based earliest-finish-time variant, and CA-List and CA-LS choose both machine and start time on the marginal (busy −

idle) cost (Supplementary Section S2, which also gives the gate for this platform; Proposition 1 covers only decisions that keep the machine type). Its hypotheses G1–G3 and G5 repeat E1–E3 and E4a; G4 tests whether CWM, on its home platform and objective, keeps less than 75% of its oracle saving. The plan excluded CaWoSched on the grounds that its input has no processor speeds. That reason was wrong, because CaWoSched keeps a fixed mapping and accepts mapped durations, so we ran it afterwards as a logged exploratory addition.

5.6 Robustness study

The main results raise four further questions: whether re-planning on newer forecasts removes the loss, whether the deadline result depends on the executor, whether reserving slack helps, and whether simpler gates do as well. A fourth plan, sealed before any of its variants was run on the confirmatory data, answers them on seeds 10 and 11 of the main study (1,200 jobs). The questions are: (Q1) receding-horizon re-planning [61] every 2 h on the newest vintage, for CA-LS and Shift with and without the gate; (Q2) the reactive executor; (Q3) a slack reserve, planning CA-LS against $D' = M + \rho(D - M)$ with $\rho \in \{0.75, 0.5\}$; (Q4) alternative gates; (Q5) the empirical coverage of the gate; and (Q6) CaWoSched and CWM with the authors' budget band. The plan states in advance how each result will be read. Because the confirmatory data had been analysed before, we report these as robustness analyses, not confirmatory tests. After the first run we found a units error that kept the re-planning variants from seeing newer forecasts; it is logged with the plan, the affected variants were re-run, and only the corrected results are reported.

5.7 Statistical analysis

Runs that share a submission time share a forecast and are correlated. The sealed analyses use a cluster bootstrap over submission times [62] (region × submission in the extensions; 2,000 resamples, fixed seed, percentile intervals). Because submissions on nearby days share weather regimes, we also report, post hoc, intervals from a bootstrap over calendar days and over ISO weeks (78 days and 14 weeks on GB; clusters are crossed with the grid in the extensions), and one-sided p-values with Holm's correction at a family level of 0.025 within each study. With 14 clusters the week bootstrap can under-cover [62], so we use it only as a sensitivity check. Supplementary Section S4 gives the cluster counts, the construction of the p-values and the test families. Means are pooled over workflows, makespan targets and seeds. Ratios of saving to the oracle saving (R, H1, E1, G1, H5) and the H3 difference pool $f \geq 1.5$, as in Eq. (3); trap rates, the gate's trap difference and retained saving (H4, E3, G3), miss rates, expected excess emission, and all points of Figs. 6a and 7 pool all four f unless a value is given per f.

6. Results

All 3,000 confirmatory jobs completed. With nominal runtimes no scheduler missed a deadline, and the safeguard kept the oracle trap rate of every carbon-aware scheduler at zero, as the sealed plan required; the same checks passed in every later study. Table 5 summarises the sealed hypotheses of the first three studies, with the robustness of each verdict to clustering by day or week.

Table 5. Pre-registered hypotheses and outcomes (95% cluster-bootstrap intervals over submission times; the last column re-checks the verdict post hoc with clusters of calendar days and weeks and with Holm's correction within each study, Section 5.7; a dash marks a hypothesis that was not supported, so there is no verdict to re-check). R: ratio of mean saving under forecasts to mean saving under the oracle; trap: run emitting more than 1% above HEFT; pp: percentage points.

Study	Hypothesis	Sealed criterion	Estimate [95% CI]	Verdict	Day / week clusters; Holm
GB	H1 relative loss (CA-LS, $f \geq 1.5$)	Upper bound of $R < 0.75$	R = 0.68 [0.62, 0.74]	Supported	Holds by day (0.745); not by week (0.755) or with Holm
GB	H2 traps (CA-LS)	Lower bound > 2% at every f	7.5% [5.5, 9.7] at f = 1.2; 13.2–15.9% at $f \geq 1.5$	Supported	Holds
GB	H3 synthetic overstates saving	Lower bound of difference > 0	+0.68 pp [−0.09, 1.48]	Not supported	—
GB	H4 gate	Trap difference < 0; retention ≥ 0.5	−8.65 pp [−10.81, −6.62]; 0.84 [0.77, 0.92]	Supported	Holds
GB	H5 CA-List / CaWoSched	H1 and H2 criteria	R = 0.68 [0.61, 0.74] / 0.73 [0.67, 0.78]; traps 6.5% [4.7, 8.5] / 7.6% [5.3, 9.9] at f = 1.2, 12.6–16.0% / 12.0–14.0% at $f \geq 1.5$	Supported / H2 only	CA-List: not by week; not with Holm
4 grids	E1 relative loss	Upper bound of $R < 0.75$	R = 0.66 [0.60, 0.72]	Supported	Holds
4 grids	E2 traps	Lower bound > 2% at every f	6.4% [4.8, 8.2] to 16.4%	Supported	Holds
4 grids	E3 gate	Trap difference < 0; retention ≥ 0.5	−8.1 pp [−9.8, −6.5]; 0.87 [0.82, 0.93]	Supported	Holds
4 grids	E4a misses vs HEFT; E4b gate	E4a: lower bound > 10 pp at every f; E4b: gated − ungated < 0	+27.6 to +43.4 pp; −21.5 pp [−23.1, −19.8]	Supported	Holds
Hetero.	G1 relative loss	Upper bound of $R < 0.75$	R = 0.85 [0.80, 0.90]	Not supported	—
Hetero.	G2 traps	Lower bound > 2% at every f	1.6% [0.6, 2.8] at f = 1.2; 4.3–5.4% at $f \geq 1.5$	Not supported	—
Hetero.	G3 gate	Trap difference < 0; retention ≥ 0.5	−2.2 pp [−3.8, −1.0]; 1.06 [1.02, 1.09]	Supported	Holds (Holm $p < 0.001$)
Hetero.	G4 CWM, native objective	Upper bound of R native < 0.75	R native = 0.32 [0.01, 0.57]	Supported	Holds; 0.58 by week; Holm p = 0.005
Hetero.	G5 misses vs HEFT	Lower bound > 10 pp at every f	+49.9 to +76.0 pp	Supported	Holds (Holm $p < 0.001$)

6.1 Issued forecasts lose about a third of the oracle saving

Fig. 4 shows the mean saving relative to HEFT under the three regimes. With oracle intensity CA-LS saved 3.2% at f = 1.2 and 21.2% at f = 3.0; with issued forecasts it saved 2.1% and 14.4%. Pooled over $f \geq 1.5$, R = 0.68 [0.62, 0.74], below the sealed threshold of 0.75 (H1). The ratio was similar at every f (0.65, 0.67, 0.68 and 0.68 at f = 1.2–3.0), so the absolute loss grew with slack, from 1.1 to 6.7 percentage points. Weighting by emissions rather than averaging per run gives ratios of 0.71–0.78. The verdict is the least robust of the sealed results. Its upper bound is 0.745 when clustering by day and 0.755 when clustering by week, and its Holm-adjusted one-sided p-value with submission-time clusters is 0.036, above the one-sided family level of 0.025 that matches the 95% intervals. The estimate itself is stable. The independent four-grid extension reproduced it pooled over grids (R = 0.66 [0.60, 0.72]; with week clusters, upper bound 0.74 and Holm-adjusted p = 0.01), although R differed widely between grids (0.19–0.85; Section 6.6). The loss was also similar across workflows (R = 0.62–0.72, Supplementary Table S1) despite very different structures, and the development year shows it in every season (R = 0.50–0.69, Supplementary Table S2). Whole-workflow Shift did as well as CA-LS under issued forecasts: its saving was within 0.5 points of CA-LS at every f, it kept a larger share of its own oracle saving (R = 0.73) and it missed fewer deadlines (Section 6.5).

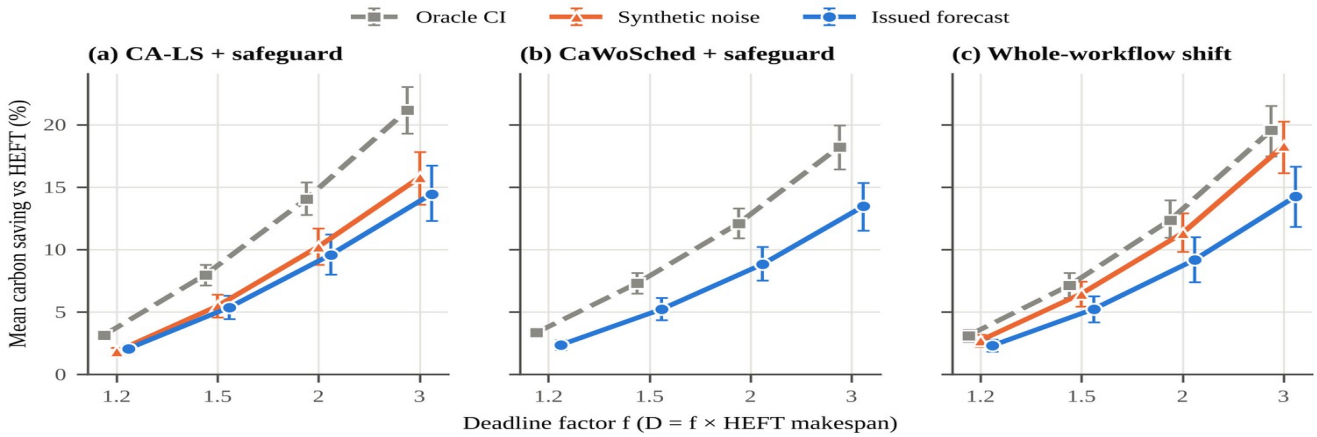


Fig. 4. Mean carbon saving relative to HEFT by deadline factor f for (a) CA-LS and (b) the official CaWoSched code, both with safeguard, and (c) whole-workflow shift, under the oracle, synthetic noise (σ = 0.20) and issued forecasts (GB, confirmatory). Whiskers: 95% cluster-bootstrap intervals. Synthetic noise was tested only for CA-LS (H3).

6.2 Forecast error turns deferrals into carbon traps

Under issued forecasts 7.5% [5.5, 9.7] of CA-LS runs were traps at f = 1.2 and 13.2–15.9% at f ≥ 1.5 (Fig. 5), so H2 holds; it is robust to every clustering. The share depends on the margin: 17.9% of runs emitted more than HEFT at all, 10.7% more than 2% above and 7.3% more than 5% above. Traps were not marginal. Their median excess was 6.2%, the 90th percentile 24.5% and the maximum 80.2%; averaged over all runs, the excess emission from traps was 1.4% of HEFT's emissions. CA-List, CaWoSched and Shift trapped as often (11.4–13.8% at the 1% margin, pooled over f). For the two schedulers of H5 the lowest rate was at f = 1.2, 6.5% [4.7, 8.5] for CA-List and 7.6% [5.3, 9.9] for CaWoSched, so both met the H2 criterion at every f (Table 5). We found no evidence that synthetic noise overstated the mean saving of CA-LS (H3 not supported: +0.68 pp [−0.09, 1.48]). In an exploratory comparison, however, i.i.d. synthetic noise misstated the risk: it produced more traps than issued forecasts for the DAG schedulers (13–21% at f ≥ 1.2) and fewer for Shift, and it overstated Shift's saving (for example 18.3% against 14.2% at f = 3.0). Temporally correlated synthetic noise might behave differently; we did not test it.

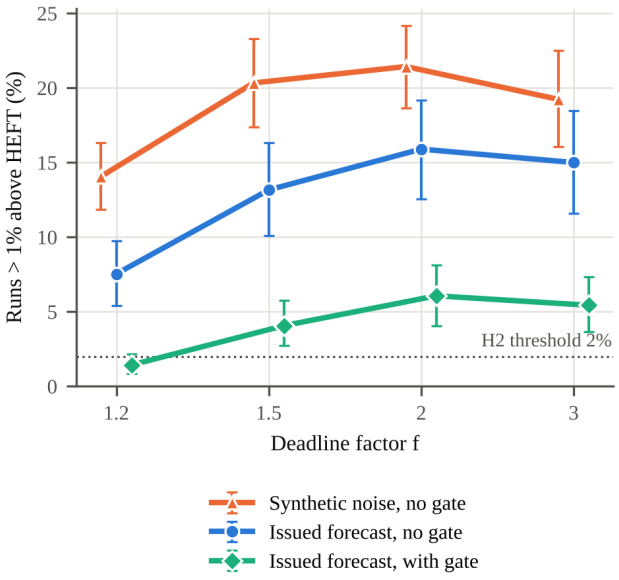


Fig. 5. Share of runs emitting more than 1% above HEFT (carbon traps) under issued forecasts and synthetic noise, by deadline factor, for CA-LS with safeguard, with and without the gate (GB, confirmatory). Whiskers: 95% cluster-bootstrap intervals.

6.3 Re-planning on newer forecasts

Re-planning every 2 h on the newest vintage (6.2 re-plans per run on average) raised the forecast-based saving of CA-LS from 10.3% to 12.3% at f ≥ 1.5, or from 72% to 86% [78%, 92%] of the static oracle saving, on the two seeds of the robustness study (Q1). Pooled over all f, it did not reduce traps: 10.8% [7.0, 14.8] of re-planned runs were traps, against 9.7% without re-planning. The sealed reading required both an upper bound of R of at least 0.9 and an upper bound of the trap rate below 2%, so re-planning does not remove the problem. With the gate, re-planning gave R = 0.73 with 4.4% traps, again a higher saving at a similar risk. For Shift, re-planning changed little (R = 0.80 against 0.78; traps 10.7% against 11.1%), because the offset is fixed once the workflow has started.

Two exploratory runs, added after the sealed results were known, separate the sources of this gain. Re-planning with oracle intensity raised the oracle saving from 14.3% to 15.7%, so the loop also optimises better than a single pass of local search. Re-planning on the submission-time vintage at every step saved 11.9%, only 0.37 points [−0.09, 0.92] less than re-planning on newer vintages. Relative to its own oracle, re-planning kept 78% [71%, 83%] of the saving, against 72% for the static scheduler. Newer forecasts therefore added little, and traps were as frequent with stale as with fresh vintages (10.2% against 10.8%). A plausible reason is that a deferral can be undone only before the deferred task starts, while a newer vintage that reveals the mistake often arrives later. Most of the error comes from the forecast available at submission, and later corrections come too late to undo it.

6.4 The difference-error gate

The gate cut the trap rate of CA-LS by 8.65 percentage points [−10.81, −6.62] while keeping 84% [77%, 92%] of the ungated saving (H4; robust to all clusterings). Expected excess emission from traps, pooled over all f, fell from 1.4% to 0.35% of HEFT's emissions. Its calibrated bounds held as intended: on the confirmatory data they covered 89.8% of the realised difference errors of the vintages actually used, against a nominal 90%, with 88.7–90.8% in every lag × horizon bin (median per submission 90.2%, 10th percentile 82%). Over task durations, as the schedulers apply them, coverage rose with the duration: 89.9% at 1 h, 92.2% at 4 h and 95.3% at 8 h (exploratory; Supplementary Fig. S2).

The robustness study compared the gate with simpler alternatives (Fig. 6a; Supplementary Table S4). On its two seeds, ungated CA-LS saved 8.31% and 9.7% of its runs were traps; with the gate it saved 6.92% [5.53, 8.52] and 3.7% [2.0, 5.7] were traps. No alternative dominated the gate, in the sense fixed in the plan: none had both a higher saving and fewer traps with paired intervals that exclude zero. The α sweep traces a smooth trade-off. At α = 0.5 the gate behaves like no gate, because the median difference error is close to zero, and α = 0.95 halves the traps again for 0.86 points of

saving. A single threshold for all lags and horizons gave up 0.79 points [0.42, 1.20] of saving at the same trap rate, so binning by lag and horizon pays for itself. The level-interval rule almost removed traps (0.6%) but kept only 58% of the ungated saving, the cost of the large threshold seen in Fig. 3c. Capping deferrals at 6 h without a gain threshold left 7.6% of runs as traps. Because it defers fewer tasks, the gate also halved deadline misses on GB (23.4% against 44.0%; Section 6.5), and on the heterogeneous platform at f ≥ 2.0 but not at f = 1.2.

A planner that uses the uncertainty of the forecast directly fared no better (exploratory). CA-LS planning on the upper 0.9-quantile of the forecast for each horizon, the same bound the level-interval rule uses, saved 7.97% with 9.1% traps. It barely differed from planning on the point forecast, because a horizon-dependent bias moves the costs of nearby windows together. Filtering individual deferrals by their difference error, as the gate does, is what reduced traps.

Controlling the risk of the whole plan did not do better either (exploratory; development data, two pilots with 20 and 40 submissions, each with a decision rule fixed in advance). A planner that minimises the mean emissions over error trajectories of past vintages, subject to a cap on the share of these scenarios in which the plan is a trap, and certifies the plan on held-out scenarios, kept the realised trap rate below its target in every setting (0.0–6.0% for targets of 5–20%). Its saving–risk points lay close to the gate's (Fig. 6b): a version that selects among candidate plans lay slightly above it (for example, 8.88% saving with 4.1% traps against 8.35% with 4.6% for the gate at α = 0.9), and a version that optimises over the scenarios lay on or below it (9.15% with 6.0% traps against 9.45% with 7.0% at α = 0.8). The differences were within the statistical uncertainty, and both pilots failed their usefulness criterion. Once the forecast at submission is fixed, the remaining choice is how much risk to accept, and a simple gate exposes that choice as well as a scenario-based optimiser.

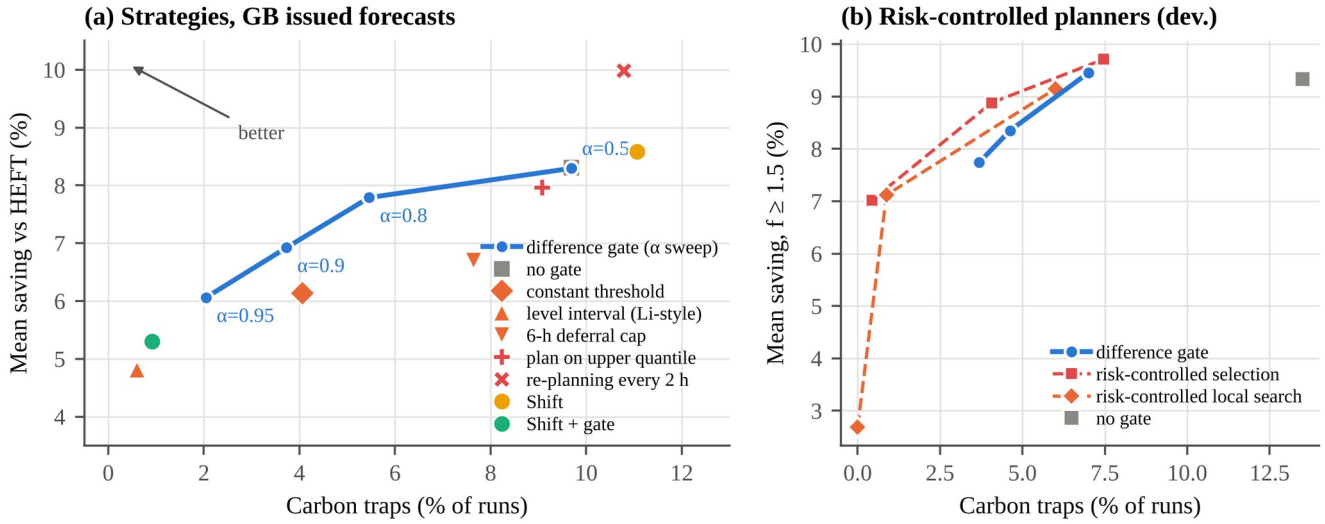


Fig. 6. Carbon saving against the share of runs that are carbon traps; points towards the upper left are better. (a) Strategies on GB issued forecasts (robustness study, seeds 10–11, all f): the difference gate for α from 0.5 to 0.95, simpler gates, an upper-quantile planner, re-planning and whole-workflow shifting. (b) The gate at α = 0.8–0.95 against two risk-controlled planners at target trap risks of 0.05–0.20 (development data, pilot 2, f ≥ 1.5; exploratory). Intervals: Supplementary Table S4.

6.5 Deadlines under runtime variability

Carbon-aware plans missed deadlines far more often than HEFT when task runtimes varied. In the main study, with issued forecasts and the static executor, CA-LS missed 31–49% of deadlines across f, against 0–1.5% for HEFT; CaWoSched and Shift missed 15–46%, and the gate roughly halved the rates (14–25%). On GB this was a secondary outcome; the sealed tests on the other grids (E4a) and the heterogeneous platform (G5) found the same pattern (Table 5). Most misses were small overruns: for CA-LS the median tardiness of a missed run was 1.5% of the deadline, the 90th percentile 5.2% and the 99th percentile 12.5%. A missed deadline is nevertheless a failure for the user. Runtime noise also reduced the realised saving slightly, from 8.31% to 7.74% for CA-LS with the static executor (robustness study).

Fig. 7 shows the two remedies tested in the robustness study. The reactive executor, which starts delayed work as soon as it is ready, reduced the gap between CA-LS and HEFT from 36–48 to 22–35 percentage points; every lower bound stayed above 17 points, so by the sealed reading the result is robust to the executor (Q2). Once an ancestor has overrun, deferred work already sits close to the deadline, and starting it early no longer helps. A slack reserve was far more effective (Q3). Planning against $D' = M + 0.75(D - M)$ cut misses from 44.0% to 3.6% (2.8% with the reactive executor) while keeping 81% of the saving (6.76% against 8.31%). The remaining gap to HEFT was 10 points at f = 1.2, 3 at f = 1.5 and none at f ≥ 2.0. With ρ = 0.5 misses fell below 1% at 59% of the saving. The reserve did not change the share of the oracle saving kept under forecasts (R = 0.73 for both reserves against 0.72 without) and reduced traps only slightly (8.9% and 7.3% against 9.7%). Reserve and gate therefore address different risks and can be combined. Shift with the gate was the most conservative option (9.5% misses, 0.9% traps, 5.3% saving). The conclusions do not depend on independent task noise (exploratory). With a job-wide runtime factor added (same marginal spread), CA-LS missed 28–43% of deadlines, 28–38 points more than HEFT, and the reserve of 25% brought misses down to 6.9%, against 2.8% for HEFT.

The size of the noise matters less than one might expect (exploratory; seed 10, Supplementary Table S5 and Fig. S3a). Even with a log-normal spread of only 0.05, CA-LS missed 28% of deadlines while HEFT missed none: a carbon-aware plan ends close to the deadline, so almost any overrun becomes a miss. With a spread of 0.4 the miss rate rose to 53% (HEFT 3%). We also resampled run-to-run variability measured in repeated executions of the same WfCommons workflows [15], whose spread ranges from 0.04 (Blast, Montage) to about 0.7 (SRASearch). Under this heavy-tailed noise HEFT itself missed 26% of deadlines and CA-LS 70%, and the 25% reserve, which removed almost all misses at spreads up to 0.2, left 42%. The reserve must therefore be sized to the runtime variability of the workflow at hand.

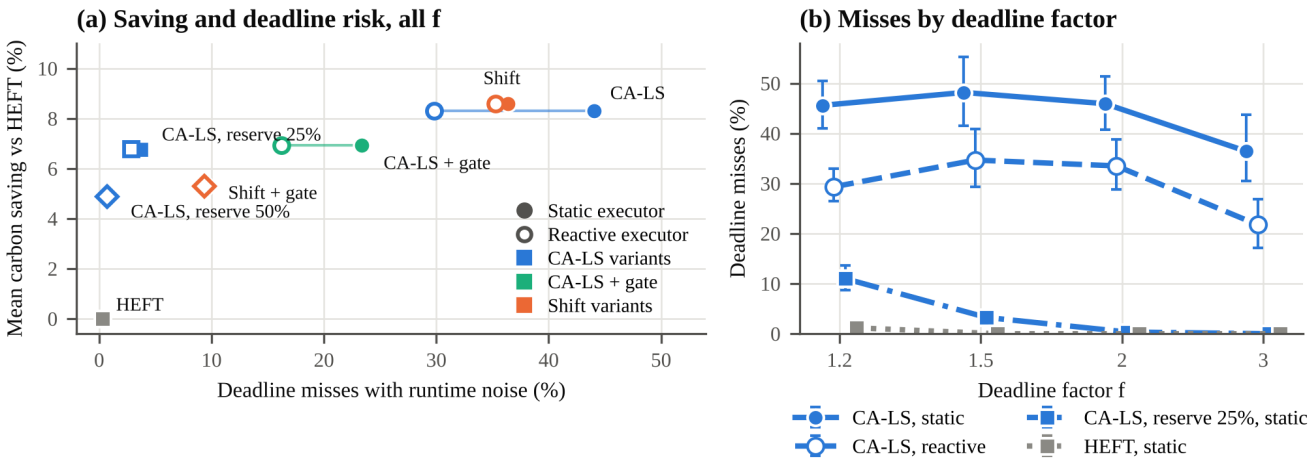


Fig. 7. Saving and deadline risk with runtime noise (robustness study, GB issued forecasts, seeds 10–11). (a) Mean saving against the share of missed deadlines, all f; filled: static executor, hollow: reactive executor; the reserves plan against $D' = M + \rho(D - M)$ with $\rho = 0.75$ and 0.5. (b) Miss rate by deadline factor. Whiskers: 95% cluster-bootstrap intervals over calendar days.

6.6 Generality across grids and forecasters

The four-grid extension (4,800 jobs) supported all four sealed hypotheses (Table 5). Pooled over grids, CA-LS kept R = 0.66 [0.60, 0.72] of its oracle saving under model forecasts; trap rates ranged from 6.4% at f = 1.2 to 16.4% at f = 3.0; the gate cut traps by 8.1 percentage points while keeping 87% of the saving; and with runtime noise CA-LS missed deadlines 27.6–43.4 points more often than HEFT, a gap the gate reduced by 21.5 points. All verdicts survive clustering by day and week and the Holm correction.

The pooled result hides a large spread. R was 0.85 [0.81, 0.89] in CISO, 0.76 [0.67, 0.84] in DE, 0.52 [0.41, 0.62] in PJM and 0.19 [−0.05, 0.36] in ERCO, where traps reached 15.5–35.3% (secondary analyses); the unweighted mean over grids is 0.58. Because the pooled R is a ratio of means, it gives more weight to grids with large savings. What explains the spread? In an exploratory analysis at the resolution of grid-months (28 points including GB, some with only six submissions), the ratio κ of forecast error to short-term variability (Eq. 4) was strongly associated with R (Spearman r_s = −0.83 [−0.92, −0.64]; Fig. 8a), and so was a rank-based measure of forecast quality (1 − Kendall τ: −0.78). The relative error was only weakly associated (WAPE: −0.10 [−0.48, 0.26]). Two caveats apply. Most of the association is between grids: within grids it was strong in GB, DE and PJM (r_s = −0.89 to −1.0), absent in CISO and reversed in ERCO, and the pooled within-grid correlation was −0.29. And because variability also raises the oracle saving, part of the association is mechanical; controlling for the standard deviation of intensity leaves r_s = −0.73 [−0.86, −0.43] for κ and −0.42 [−0.71, 0.02] for WAPE. PJM, for example, has the most accurate forecasts in relative terms (WAPE 3–6%) but a flat intensity profile, so even small errors mis-rank its windows. Grids with κ near or above one kept little of the oracle saving. Fig. 8b shows that the gate reduced traps on every grid.

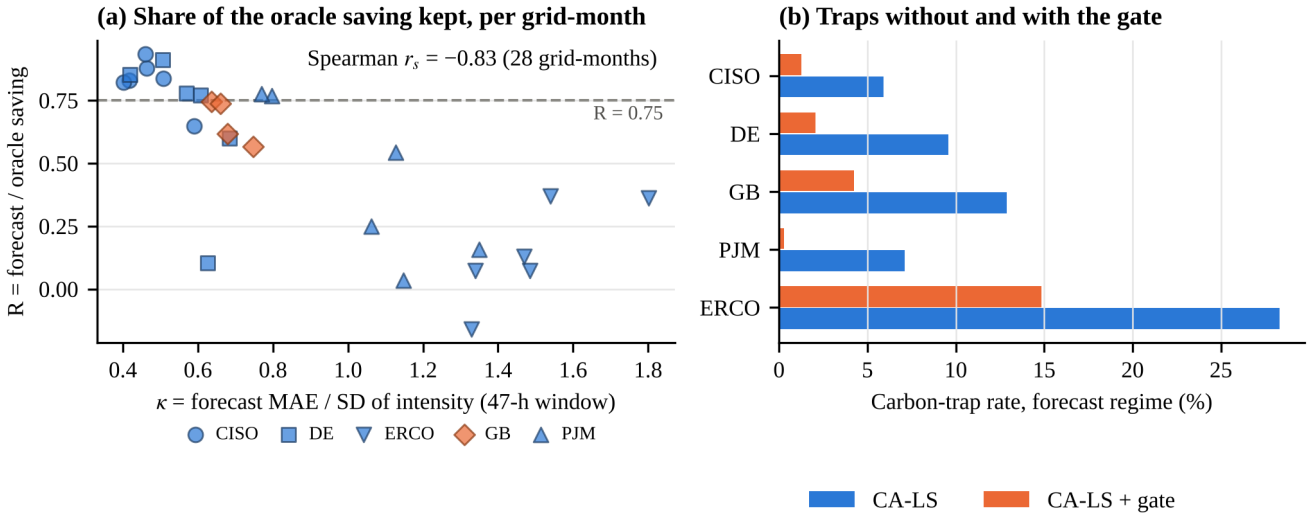


Fig. 8. Forecast loss across grids (exploratory). (a) Share of the oracle saving kept by CA-LS (f ≥ 1.5) per grid-month against κ, the forecast MAE over the standard deviation of intensity in the same 47-h window; GB: issued vintages (2026), other grids: CarbonCast (2021). All 28 grid-months are shown, semi-transparent where they overlap. (b) Trap rate without and with the gate, all f.

The official baselines told the same story on their own objective. Under the oracle, safeguarded CaWoSched reduced brown energy above the green budget by 39–44% relative to HEFT and safeguarded CWM by about 22% (the safeguard compares planned emissions, Section 4.2; brown energy is then evaluated on the final plan); with model forecasts these reductions fell to 28–29% and 17–18% on CISO and DE, 12% and 7% on PJM, and −13% and −7% on ERCO. Shift stayed within 0.3 percentage points of CA-LS on every grid.

6.7 Platform sensitivity and the official schedulers

Fig. 9 summarises this section. On the heterogeneous platform with idle power (1,600 jobs), two of the five sealed hypotheses failed.

CA-LS kept R = 0.85 [0.80, 0.90] of its oracle emission saving (G1), and its trap rate was 1.6% [0.6, 2.8] at f = 1.2 and 4.3–5.4% at f ≥ 1.5, with lower bounds of 1.7–2.5% (G2). The gate still cut traps and here even raised the saving (G3; retention 1.06, so the deferrals it blocked had lost carbon on average), CWM kept only R = 0.32 [0.01, 0.57] of its oracle reduction in brown energy (G4), and carbon-aware plans missed deadlines 49.9–76.0 points more often than HEFT (G5). The three supported verdicts held with clusters of calendar days and weeks and after Holm's correction (Table 5); the widest interval, that of G4, had an upper bound of 0.58 by week.

The failures have a clear cause. On this platform much of the saving comes from placing work on the more efficient machine type, which needs no forecast. Planned on a flat carbon signal, CA-LS saved 6.05% on a subset of 1,600 runs per regime (five submissions per grid, all f), against 7.00% with the oracle and 6.18% with model forecasts (exploratory; 95% intervals from 20 region × submission clusters). Carbon timing therefore added only about one percentage point under the oracle, 0.94 [−0.03, 2.14], too little to say how much of it forecasts keep: the share of the timing gain kept, $(S_{forecast} - S_{flat}) / (S_{oracle} - S_{flat})$, was 0.14 with an interval of [−4.1, 2.2]. In DE and PJM, planning on a flat signal saved as much as planning on the oracle (Fig. 9a), so there the forecast had almost nothing to add. CaWoSched, which keeps the HEFT mapping and can only re-time work, kept R = 0.71, with trap rates of 2.3–23.4% across grids, much as on the homogeneous platform (exploratory). The ratio R thus depends on how much of a scheduler's saving comes from timing: G1 and G2 failed because R here mixes an energy-aware mapping, which needs no forecast, with carbon timing, not because forecasts did better.

With our intensity-derived budget, CWM did not reduce emissions on either platform: on the homogeneous platform its safeguarded emission saving was 0.4–3.5% under model forecasts (0.7–3.7% under the oracle), and on its own processor types raw CWM saved −1.9% to +0.3% under the oracle. Its native objective did improve (22% under the oracle on the homogeneous platform), and the 69% reported by its authors is a reduction of that objective under synthetic budgets. The gap is therefore between objectives: brown energy above a budget derived from intensity does not track emissions computed from intensity, at least with our budget mapping (Supplementary Section S1). With the authors' own budget band (frac = 0.2 and 0.4; robustness study, seeds 10–11) the picture was unchanged. CaWoSched with the safeguard saved 9.4–9.5% under the oracle and 7.2–7.3% under issued forecasts (R = 0.76–0.77; traps 9.1–9.2%), slightly less than with frac = 1.0 on the same seeds (10.2% and 7.9%). Raw CWM emitted more than HEFT on average with every band (−7.1% to −0.2% under the oracle), and with the safeguard it saved 4.3–4.6%.

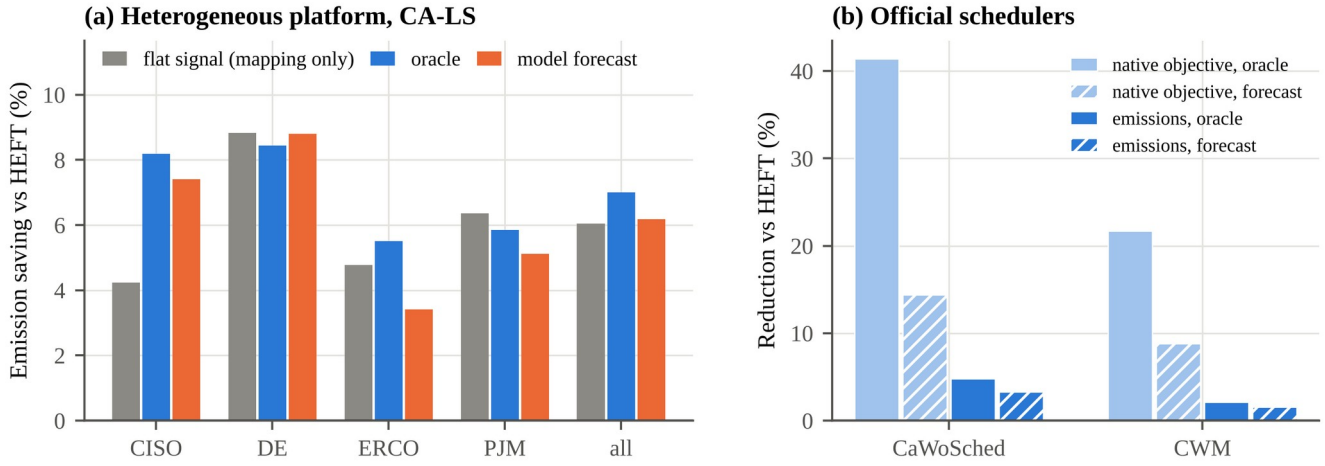


Fig. 9. (a) Heterogeneous platform: emission saving of CA-LS relative to HEFT when planned on a flat signal (mapping only), on the oracle and on model forecasts (exploratory; 400 runs per grid and regime). (b) Safeguarded official schedulers on the homogeneous platform (four grids pooled): reduction of their native objective (brown energy above the green budget) and of emissions.

6.8 Scheduling overhead

All schedulers are fast enough to run at submission time. On one core (Python implementation; 60 runs per scheduler over the 20 workflow instances of seed 10, f = 2.0), HEFT took a median of 0.012 s, Shift 0.005 s, CA-List 0.031 s and CA-LS 0.070 s (95th percentile 0.24 s). Calibrating the gate from seven days of vintages took a median of 0.23 s (95th percentile 0.81 s) and is needed once per submission, and gated CA-LS took 0.12 s. These times are negligible compared with workflows that run for hours.

7. Discussion

7.1 Why forecasts hurt workflow deferral

Each deferral pays off only if the forecast ranks the two windows correctly, so the error that matters is the error of a difference between windows, not of a level. A workflow plan makes many deferrals on one vintage; their errors are correlated rather than averaged out, and the safeguard, which relies on the same vintage, cannot detect them. Proposition 1 bounds the risk of each deferral (exactly for single-window tasks, approximately for longer ones) but not of the plan, which is why a gate reduces traps without removing them. Three further observations point to the forecast at submission as the bottleneck. Re-planning on newer vintages improved the optimisation but added little information (Section 6.3); planning on a pessimistic forecast changed little, because a horizon-dependent bias shifts nearby windows together; and a planner that controls the trap risk of the whole plan over error scenarios did no better than the gate (Section 6.4). Across grids, how much of the saving survived was associated with the forecast error relative to the short-term variability of intensity more than with the relative error (exploratory, Fig. 8a). This agrees with the observation by Joseph et al. [14] that ranking quality matters more than point accuracy, and it suggests that accuracy metrics such as MAPE [11,45] are a poor guide to the value of a forecast for scheduling.

7.2 Deadlines

A carbon-aware plan spends the slack that, in a carbon-agnostic plan, absorbs overruns [46,47]. With moderate runtime noise this raised deadline misses by 28–49 percentage points on the homogeneous platform (GB and the four-grid extension), and a work-conserving executor reduced but did not remove the gap (Section 6.5). For moderate noise the remedy is cheap. Planning against a deadline that keeps a quarter of the slack in reserve cut misses from 44% to under 4% while keeping about four-fifths of the saving, and a work-conserving executor was no substitute for it [E]. Under the heavy-tailed variability measured in real executions, a quarter of the slack was no longer enough (Section 6.5), so the

reserve has to follow the workflow. Evaluations of carbon-aware schedulers should report deadline misses under runtime variation alongside savings.

7.3 Recommendations

We separate recommendations backed by sealed tests [C] from those based on exploratory or robustness analyses [E]. (1) Evaluate carbon-aware schedulers on forecasts that were actually issued, not on hindsight [C]; i.i.d. synthetic noise misstated the trap risk [E]. (2) Gate deferrals by a calibrated bound on forecast-difference errors; it reduced traps in every sealed test [C] and on every grid [E], and simpler gates did not dominate it [E]. (3) Reserve part of the slack, sized to the runtime variability of the workflow [E]. (4) Before deploying, measure the forecast error relative to the short-term variability of intensity (κ); where κ approaches one, expected gains are small and traps frequent [E]. (5) Compare against whole-workflow shifting: on homogeneous platforms it matched the forecast-based saving of task-level scheduling, with fewer deadline misses, and is far simpler [E]; on our heterogeneous platform, energy-aware mapping mattered more than carbon timing [E]. (6) Report the split of savings into an energy part and a timing part, since only the latter depends on forecasts [E].

7.4 Threats to validity

Internal. All schedulers run in one simulator at one-minute resolution without data-transfer time. On the homogeneous platform idle power is not attributed, which assumes idle machines are powered down or serve other work. If they serve other work, deferral may displace it, an effect we do not model [30]. The official codes required adapters (Supplementary Section S1). Local search uses a fixed budget of 2|V| moves; repeated optimisation raised the oracle saving by 1.4 points (Section 6.3), so absolute savings are conservative, although the shares kept under forecasts changed little. Our budget mapping was chosen because it gave the tools their best oracle saving, and the authors' narrower band was also tested. The gate's per-decision bound is exact only for single-window deferrals that keep the machine type; for longer tasks it is approximate, and for changes of machine type on the heterogeneous platform the gate is a heuristic (Section 4.3, Supplementary Section S2). **External.** Issued vintages were available for one grid and one season. The other grids use model forecasts from 2021, which may be better or worse than operator forecasts. The development year suggests the loss is present in every season, but it uses an earlier configuration without the safeguard. The instances are WfCommons workflows, and most are limited by capacity rather than by their critical path; per-workflow results did not depend on structure. On clusters of 32 and 64 machines (exploratory, seed 10, Supplementary Table S6 and Fig. S3), CA-LS kept a similar share of its oracle saving (R = 0.79 against 0.80 on 8 machines), traps became more frequent (7.4% and 9.7% against 5.5%) and the gate still cut them to 2.2–2.5%; the 25% reserve was less effective there (16–28% misses), because HEFT itself started to miss deadlines at f = 1.2. Communication-heavy workflows may behave differently. We use the average intensity published by the operators, and the realised values are themselves estimates; marginal signals can lead to different decisions [20,63], as can location- versus market-based accounting [64], and only operational emissions are counted [65]. **Construct.** Traps use a 1% margin and R is a ratio of means; we therefore report other margins, the expected excess emission, R per f, absolute losses and native objectives. **Statistical.** The sealed analyses cluster by submission time. Clustering by week and correcting for multiple tests weakens one verdict (H1 on GB, and with it the H1 part of H5 for CA-List), which the four-grid extension confirms robustly; the week bootstrap on GB rests on only 14 clusters. Pooled results of the extension average over very different grids, and the grid-month analysis rests on few submissions per month. The synthetic-noise regime is i.i.d., and runtime noise is i.i.d. per task except in one exploratory check. Two of five hypotheses on the heterogeneous platform and one of five in the main study were not supported, and several analyses are exploratory; we label them as such.

8. Conclusion

Carbon-aware workflow scheduling has been evaluated almost entirely with perfect foresight. Replayed against forecasts that were actually issued, a carbon-aware DAG scheduler kept about two-thirds of its oracle saving on GB, and about one plan in seven with moderate slack emitted more than a carbon-agnostic schedule. The official CaWoSched code lost a similar share, and CWM's native objective did not track emissions under our budget mapping. Across four further grids the share kept ranged from a fifth to most of the saving. A simple gate calibrated on errors of forecast differences reduced traps on every grid at a modest cost in saving, and neither re-planning on newer forecasts, nor planning on an upper forecast quantile, nor controlling the risk of the whole plan over error scenarios reduced them further. Whole-workflow shifting did as well as task-level scheduling, and it is a baseline that every evaluation should include. On a heterogeneous platform, most of the saving came from energy-aware mapping, which needs no forecast. Carbon-aware plans also spend the slack that protects deadlines, and a reserve sized to the runtime variability of the workflow restored most of that protection. Across grids, the value of a forecast for scheduling was associated with its error relative to the short-term variability of intensity more than with its relative accuracy, which suggests judging forecasters by how well they rank windows. The evidence has limits: issued vintages for one grid and season, model forecasts for the others, simulated execution, and WfCommons workflows on small clusters. A pre-registered winter replication on GB is planned. Future work should replay issued forecasts from more grids and seasons, consider marginal intensity and shared clusters, and design schedulers that reason about the errors of forecast differences rather than levels. All plans, code and results are public, so others can repeat the evaluation with their own schedulers and forecasts.

Declarations

CRediT authorship contribution statement. Hemn Khezri: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft. **Mohammad Trik:** Conceptualization, Validation, Supervision, Writing – review & editing.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability. Code, sealed plans, deviation logs and raw results are available at [repository URL and Zenodo DOI]. The GB forecast vintages come from the public archive nmpowell/carbon-intensity-forecast-tracking (commit 578de5b) [55], and the CarbonCast forecasts from the public CarbonCast repository [60].

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work the authors used Claude (Anthropic; Claude Opus 5.5 and Claude Sonnet 5) to help write and run the simulation and analysis code, to write the code that draws all figures, including the explanatory diagrams of Figs. 1 and 3a, and to draft and edit parts of the text and improve its language. After using this tool, the authors reviewed and edited the code, results and content as needed and take full responsibility for the content of the published article.

References

[1] E. Masanet, A. Shehabi, N. Lei, S. Smith, J. Koomey, Recalibrating global data center energy-use estimates, Science 367 (6481) (2020) 984–986. https://doi.org/10.1126/science.aba3758.

[2] International Energy Agency, Energy and AI, Technical report, Paris, 2025. https://www.iea.org/reports/energy-and-ai.

[3] European Commission, Commission Delegated Regulation (EU) 2024/1364 of 14 March 2024 on the first phase of the establishment of a

common Union rating scheme for data centres, Official Journal of the European Union, 2024. https://eur-lex.europa.eu/eli/reg_del/2024/1364/oj.

[4] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, L. Thamsen, Let's wait awhile: how temporal workload shifting can reduce carbon emissions in the cloud, in: Proceedings of the 22nd International Middleware Conference, ACM, 2021, pp. 260–272. https://doi.org/10.1145/3464298.3493399.

[5] A. Radovanovic, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D.-Y. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, W. Cirne, Carbon-Aware Computing for Datacenters, IEEE Trans. Power Syst. 38 (2) (2023) 1270–1280. https://doi.org/10.1109/TPWRS.2022.3173250.

[6] D. Schweisgut, A. Benoit, Y. Robert, H. Meyerhenke, Carbon-Aware Workflow Scheduling with Fixed Mapping and Deadline Constraint, in: Proceedings of the 54th International Conference on Parallel Processing (ICPP), 2025, pp. 627–637. https://doi.org/10.1145/3754598.3754673.

[7] D. Schweisgut, A. Benoit, Y. Robert, H. Meyerhenke, Carbon-Aware Mapping and Scheduling for Deadline-Constrained Workflows, in: M. Torquati, G. Mencagli, V. Cardellini, A. Antelmi, D. Medić (Eds.), Euro-Par 2026: Parallel Processing, Lecture Notes in Computer Science, vol. 16782, Springer Nature Switzerland, Cham, 2026, pp. 270–284. https://doi.org/10.1007/978-3-032-35251-4_19.

[8] A. Lechowicz, R. Shenoy, N. Bashir, M. Hajiesmaili, A. Wierman, C. Delimitrou, Carbon- and Precedence-Aware Scheduling for Data Processing Clusters, in: Proceedings of the ACM SIGCOMM 2025 Conference, ACM, 2025, pp. 1241–1244. https://doi.org/10.1145/3718958.3750478.

[9] K. West, Y. Moawad, F. Lehmann, V. Bountris, U. Leser, Y. Elkhatib, L. Thamsen, A systematic evaluation of the potential of carbon-aware execution for scientific workflows, Future Gener. Comput. Syst. 182 (2026) 108453. https://doi.org/10.1016/j.future.2026.108453.

[10] A.R.W. Bruce, L. Ruff, J. Kelloway, F. MacMillan, A. Rogers, Carbon Intensity Forecast Methodology, Technical report, National Grid Electricity System Operator, 2021. https://github.com/carbon-intensity/methodology.

[11] D. Maji, P. Shenoy, R.K. Sitaraman, Multi-day Forecasting of Electric Grid Carbon Intensity using Machine Learning, ACM SIGENERGY Energy Inform. Rev. 3 (2) (2023) 19–33. https://doi.org/10.1145/3607114.3607117.

[12] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, P. Shenoy, On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud, in: Proceedings of the Nineteenth European Conference on Computer Systems, ACM, 2024, pp. 924–941. https://doi.org/10.1145/3627703.3650079.

[13] A. Li, S. Liu, Y. Ding, Uncertainty-Aware Decarbonization for Datacenters, ACM SIGENERGY Energy Inform. Rev. 4 (5) (2024). https://doi.org/10.1145/3727200.3727221.

[14] M. Joseph, T. Savadi, A. Souza, LiteCast: A Lightweight Forecaster for Carbon Optimizations, arXiv preprint arXiv:2511.06187, 2025. https://doi.org/10.48550/arXiv.2511.06187.

[15] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, R. Ferreira da Silva, WfCommons: A Framework for Enabling Scientific Workflow Research and Development, Future Gener. Comput. Syst. 128 (2022) 16–27. https://doi.org/10.1016/j.future.2021.09.043.

[16] Z.-H. Liu, M. Lin, A. Wierman, S. Low, L. Andrew, Greening geographical load balancing, in: Proceedings of the ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems, 2011, pp. 233–244. https://doi.org/10.1145/1993744.1993767.

[17] P.X. Gao, A.R. Curtis, B. Wong, S. Keshav, It's not easy being green, in: Proceedings of the ACM SIGCOMM 2012 Conference, 2012, pp. 211–222. https://doi.org/10.1145/2342356.2342398.

[18] Z. Zhou, F. Liu, R. Zou, J. Liu, H. Xu, H. Jin, Carbon-Aware Online Control of Geo-Distributed Cloud Services, IEEE Trans. Parallel Distrib. Syst. 27 (9) (2016) 2506–2519. https://doi.org/10.1109/TPDS.2015.2504978.

[19] Í. Goiri, M.E. Haque, K. Le, R. Beauchea, T.D. Nguyen, J. Guitart, J. Torres, R. Bianchini, Matching renewable energy supply and demand in green datacenters, Ad Hoc Netw. 25 (2015) 520–534. https://doi.org/10.1016/j.adhoc.2014.11.012.

[20] T. Sukprasert, N. Bashir, A. Souza, D. Irwin, P.J. Shenoy, On the Implications of Choosing Average versus Marginal Carbon Intensity Signals on Carbon-aware Optimizations, in: Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems (e-Energy), 2024, pp. 422–427. https://doi.org/10.1145/3632775.3661953.

[21] W.A. Hanafy, Q. Liang, N. Bashir, D.E. Irwin, P.J. Shenoy, CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency, Proc. ACM Meas. Anal. Comput. Syst. 7 (3) (2023) 57:1–57:28. https://doi.org/10.1145/3626788.

[22] T.B. Hewage, S. Ilager, M.A. Rodriguez, R. Buyya, A Framework for Carbon-Aware Real-Time Workload Management in Clouds Using Renewables-Driven Cores, IEEE Trans. Comput. 74 (8) (2025) 2757–2771. https://doi.org/10.1109/TC.2025.3571495.

[23] T. Yang, H. Jiang, Y. Hou, Y. Geng, Carbon Management of Multi-Datacenter Based On Spatio-Temporal Task Migration, IEEE Trans. Cloud Comput. 11 (1) (2023) 1078–1090. https://doi.org/10.1109/TCC.2021.3130644.

[24] W.A. Hanafy, Q. Liang, N. Bashir, A. Souza, D. Irwin, P.J. Shenoy, Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 3, 2024, pp. 479–496. https://doi.org/10.1145/3620666.3651374.

[25] L. Wu, W.A. Hanafy, A. Souza, K. Nguyen, J. Harkes, D. Irwin, M. Satyanarayanan, P. Shenoy, CarbonEdge: Leveraging Mesoscale Spatial Carbon-Intensity Variations for Low Carbon Edge Computing, in: Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing, ACM, 2025, pp. 1–13. https://doi.org/10.1145/3731545.3731576.

[26] Y. Son, U. Gupta, A. McCrabb, Y.G. Kim, V. Bertacco, D. Brooks, C.-J. Wu, GreenScale: Carbon Optimization for Edge Computing, IEEE Internet Things J. 12 (16) (2025) 32379–32393. https://doi.org/10.1109/JIOT.2025.3555153.

[27] G. Perin, M. Berno, T. Erseghe, M. Rossi, Towards Sustainable Edge Computing Through Renewable Energy Resources and Online, Distributed and Predictive Scheduling, IEEE Trans. Netw. Serv. Manag. 19 (1) (2022) 306–321. https://doi.org/10.1109/TNSM.2021.3112796.

[28] N. Tirel, P. Roose, S. Ilarri, A. Noureddine, O.L. Goaër, Workload Shifting Techniques: From Digital Inebriation to Sobriety, ACM Comput. Surv. 58 (5) (2026) 129:1–129:36. https://doi.org/10.1145/3769301.

[29] S.S. Gill, R. Buyya, A Taxonomy and Future Directions for Sustainable Cloud Computing: 360 Degree View, ACM Comput. Surv. 51 (5) (2019) 1–33. https://doi.org/10.1145/3241038.

[30] J.R.L. Senga, S. Wang, C.R. Knittel, Flexible data centers reduce power system costs but can increase emissions, iScience 29 (7) (2026) 116497. https://doi.org/10.1016/j.isci.2026.116497.

[31] A. Lechowicz, N. Christianson, J. Zuo, N. Bashir, M. Hajiesmaili, A. Wierman, P. Shenoy, The Online Pause and Resume Problem: Optimal Algorithms and An Application to Carbon-Aware Load Shifting, Proc. ACM Meas. Anal. Comput. Syst. 7 (3) (2023) 45:1–45:32. https://doi.org/10.1145/3626776.

[32] R. Bostandoost, A. Lechowicz, W.A. Hanafy, N. Bashir, P. Shenoy, M. Hajiesmaili, LACS: Learning-Augmented Algorithms for Carbon-Aware Resource Scaling with Uncertain Demand, in: The 15th ACM International Conference on Future and Sustainable Energy Systems, ACM, 2024, pp. 27–45. https://doi.org/10.1145/3632775.3661942.

[33] A. Lechowicz, N. Christianson, B. Sun, N. Bashir, M. Hajiesmaili, A. Wierman, P. Shenoy, Learning-Augmented Competitive Algorithms for Spatiotemporal Online Allocation with Deadline Constraints, Proc. ACM Meas. Anal. Comput. Syst. 9 (1) (2025) 8:1–8:49. https://doi.org/10.1145/3711701.

[34] E. Johnson, A. Lechowicz, M. Hajiesmaili, Signal-Aware Workload Shifting Algorithms with Uncertainty-Quantified Predictors, arXiv preprint arXiv:2509.26511, 2025. https://doi.org/10.48550/arXiv.2509.26511.

[35] R. Bostandoost, W.A. Hanafy, A. Lechowicz, N. Bashir, P. Shenoy, M. Hajiesmaili, Data-driven Algorithm Selection for Carbon-Aware Scheduling, ACM SIGENERGY Energy Inform. Rev. 4 (5) (2024) 148–153. https://doi.org/10.1145/3727200.3727222.

[36] A. Sohrabbeig, O. Ardakanian, P. Musilek, Learning to Reduce Data Center Carbon Emissions via Forecast-Integrated Workload Scheduling, in: 2026 IEEE Power & Energy Society General Meeting (PESGM), 2026. https://doi.org/10.1109/PESGM58988.2026.11693224.

[37] S. Abrishami, M. Naghibzadeh, D.H.J. Epema, Deadline-constrained workflow scheduling algorithms for Infrastructure as a Service Clouds, Future Gener. Comput. Syst. 29 (1) (2013) 158–169. https://doi.org/10.1016/j.future.2012.05.004.

[38] M.A. Rodriguez, R. Buyya, Deadline Based Resource Provisioning and Scheduling Algorithm for Scientific Workflows on Clouds, IEEE Trans. Cloud Comput. 2 (2) (2014) 222–235. https://doi.org/10.1109/TCC.2014.2314655.

[39] M.A. Rodriguez, R. Buyya, A taxonomy and survey on scheduling algorithms for scientific workflows in IaaS cloud computing environments, Concurr. Comput. Pract. Exp. 29 (8) (2017) e4041. https://doi.org/10.1002/cpe.4041.

[40] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, IEEE Trans. Parallel Distrib. Syst. 13 (3) (2002) 260–274. https://doi.org/10.1109/71.993206.

[41] S. Garg, C.S. Yeo, A. Anandasivam, R. Buyya, Environment-conscious scheduling of HPC applications on distributed Cloud-oriented data centers, J. Parallel Distrib. Comput. 71 (6) (2011) 732–749. https://doi.org/10.1016/j.jpdc.2010.04.004.

[42] M. Xu, R. Buyya, Managing renewable energy and carbon footprint in multi-cloud computing environments, J. Parallel Distrib. Comput. 135 (2020) 191–202. https://doi.org/10.1016/j.jpdc.2019.09.015.

[43] R. Bostandoost, A. Lechowicz, W.A. Hanafy, P. Shenoy, M. Hajiesmaili, Quantifying the Carbon Reduction of DAG Workloads: A Job Shop Scheduling Perspective, in: Proceedings of the 2nd International Workshop on Systems and Methods for Sustainable Large-Scale AI (GreenSys), ACM, 2026, pp. 1–6. https://doi.org/10.1145/3802973.3804454.

[44] D. Maji, R.K. Sitaraman, P. Shenoy, DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids Using Machine Learning, in: Proceedings of the Thirteenth ACM International Conference on Future Energy Systems (e-Energy), 2022, pp. 188–192. https://doi.org/10.1145/3538637.3538849.

[45] L. Yan, L. Wang, S. Liu, Y. Ding, EnsembleCI: Ensemble Learning for Carbon Intensity Forecasting, in: Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems, ACM, 2025, pp. 208–212. https://doi.org/10.1145/3679240.3734630.

[46] L.-C. Canon, E. Jeannot, Evaluation and Optimization of the Robustness of DAG Schedules in Heterogeneous Environments, IEEE Trans. Parallel Distrib. Syst. 21 (4) (2010) 532–546. https://doi.org/10.1109/TPDS.2009.84.

[47] Z. Shi, E. Jeannot, J.J. Dongarra, Robust task scheduling in non-deterministic heterogeneous computing systems, in: 2006 IEEE International Conference on Cluster Computing (Cluster), 2006. https://doi.org/10.1109/CLUSTR.2006.311868.

[48] M. Lombardi, M. Milano, L. Benini, Robust Scheduling of Task Graphs under Execution Time Uncertainty, IEEE Trans. Comput. 62 (1) (2013) 98–111. https://doi.org/10.1109/TC.2011.203.

[49] A. Taghinezhad-Niar, S. Pashazadeh, J. Taheri, QoS-aware online scheduling of multiple workflows under task execution time uncertainty in clouds, Clust. Comput. 25 (6) (2022) 3767–3784. https://doi.org/10.1007/s10586-022-03600-8.

[50] A. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster, Conformal Risk Control, in: B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, Y. Sun (Eds.), International Conference on Learning Representations, 2024, pp. 55198–55218. https://proceedings.iclr.cc/paper_files/paper/2024/file/f3549ef9b5ff520a7e41ff3cc306ab2b-Paper-Conference.pdf.

[51] C. Xu, Y. Xie, Conformal prediction interval for dynamic time-series, in: Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR, vol. 139, 2021, pp. 11559–11569. https://proceedings.mlr.press/v139/xu21h.html.

[52] I. Gibbs, E. Candès, Adaptive Conformal Inference Under Distribution Shift, in: M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, J.W. Vaughan (Eds.), Advances in Neural Information Processing Systems, vol. 34, Curran Associates, Inc., 2021, pp. 1660–1672. https://proceedings.neurips.cc/paper_files/paper/2021/file/0d441de75945e5acbc865406fc9a2559-Paper.pdf.

[53] D. Schweisgut, CaWoSched [software], GitHub, commit dfc8e49, 2026. https://github.com/KIT-EAE/CaWoSched.

[54] D. Schweisgut, CWM [software], GitHub, commit 50e9ef5, 2026. https://github.com/KIT-EAE/CWM.

[55] [dataset] N. Powell, carbon-intensity-forecast-tracking, GitHub, commit 578de5b, 2026. https://github.com/nmpowell/carbon-intensity-forecast-tracking.

[56] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, K. Vahi, Characterizing and Profiling Scientific Workflows, Future Gener. Comput. Syst. 29 (3) (2013) 682–692. https://doi.org/10.1016/j.future.2012.08.015.

[57] WfCommons project, WfCommons, version 1.6 [software], GitHub, n.d. https://github.com/wfcommons/WfCommons.

[58] T. Hoefler, R. Belli, Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses when Reporting Performance Results, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC15), 2015, pp. 73:1–73:12. https://doi.org/10.1145/2807591.2807644.

[59] B.A. Nosek, C.R. Ebersole, A.C. DeHaven, D.T. Mellor, The preregistration revolution, Proc. Natl. Acad. Sci. U.S.A. 115 (11) (2018) 2600–2606. https://doi.org/10.1073/pnas.1708274114.

[60] CarbonFirst group, CarbonCast [software], GitHub, commit 12d6989, 2024. https://github.com/carbonfirst/CarbonCast.

[61] W. Zhang, L. Roald, V. Zavala, Exploring the Impacts of Power Grid Signals on Data Center Operations Using a Receding-Horizon Scheduling Model, IEEE Trans. Sustain. Comput. 8 (2) (2023) 245–256. https://doi.org/10.1109/TSUSC.2022.3224668.

[62] A.C. Cameron, J.B. Gelbach, D.L. Miller, Bootstrap-Based Improvements for Inference with Clustered Errors, Rev. Econ. Stat. 90 (3) (2008) 414–427. https://doi.org/10.1162/rest.90.3.414.

[63] P. Wiesner, O. Kao, Moving Beyond Marginal Carbon Intensity: A Poor Metric for Both Carbon Accounting and Grid Flexibility, ACM SIGMETRICS Perform. Eval. Rev. 53 (2) (2025) 108–111. https://doi.org/10.1145/3764944.3764969.

[64] D. Maji, N. Bashir, D. Irwin, P.J. Shenoy, R. Sitaraman, The Green Mirage: Impact of Location- and Market-based Carbon Intensity Estimation on Carbon Optimization Efficacy, in: Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems (e-Energy), 2024, pp. 13–24. https://doi.org/10.1145/3632775.3639587.

[65] N. Bashir, V. Gohil, A.B. Subramanya, M. Shahrad, D. Irwin, E. Olivetti, C. Delimitrou, The Sunk Carbon Fallacy: Rethinking Carbon Footprint Metrics for Effective Carbon-Aware Scheduling, in: Proceedings of the ACM Symposium on Cloud Computing, ACM, 2024, pp. 542–551. https://doi.org/10.1145/3698038.3698542.

Appendix A. Deviations from the sealed plans

(1) Main study: the scheduler horizon was extended to D + T and the planning signal padded with its last value, because intermediate attempts inside the deadline repair could place tasks beyond the 47-h vintage before being rejected; valid plans never use the padded values. The partial output of the failed launch was deleted unread. (2) Main study: the run was split by seed to fit a two-hour process limit; the design is unchanged. (3) Main study: the CaWoSched adapter's documentation wrongly said its budget mapping followed CWM's; the code was unchanged, and Supplementary Section S1 states the mapping used. (4) Platform study: CaWoSched was added as an exploratory analysis after the results, because the reason for excluding it was wrong (Section 5.5).